

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ  
«МИФИ»

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕНЕТИЧЕСИКХ СИСТЕМ

КАФЕДРА «МАШИННОЕ ОБУЧЕНИЕ»

На правах рукописи

УДК \_\_\_\_\_

Пападык Всеволод Дмитриевич

**«Построение и применение композитных векторных представлений  
текста для многоклассовой классификации текстов»**

Выпускная квалификационная работа магистра

Направление подготовки 01.04.02

«Прикладная математика и информатика»

Выпускная  
квалификационная работа  
защищена

«\_\_»\_\_\_\_\_ 2026

г.

Оценка

\_\_\_\_\_

Секретарь

ГЭЖ

---

г. Москва

2026

## РЕФЕРАТ

### КОМПОЗИТНЫЕ ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ, МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ ТЕКСТОВ, АУГМЕНТАЦИЯ ДАННЫХ, TF-IDF, RUBERT

Объектом исследования являются методы автоматической многоклассовой классификации текстов в условиях ограниченного и несбалансированного корпоративного датасета.

Целью работы является исследование и реализация различных методов построения векторных представлений текста, включая подготовку к использованию композитных эмбеддингов, и оценка их влияния на качество многоклассовой классификации с учётом совместной предобработки и аугментации данных.

В процессе работы проведён разведочный анализ исходных данных, выявлены структура и дисбаланс классов, разработаны и реализованы процедуры предобработки и очистки текстов. Реализованы два сценария текстовой аугментации для малочисленных классов: перефразирование и обратный перевод, позволяющие выровнять распределение классов и увеличить вариативность формулировок.

Построены и исследованы модели классификации на основе классического пайплайна TF-IDF + LinearSVC и нейросетевой модели ruBERT, обученные на исходном и аугментированных наборах данных. Выполнено сравнение моделей по набору метрик (ассурасу, balanced ассурасу, макро- и взвешенная F1-мера), а также проведён анализ ошибок и трудных классов. На основе полученных результатов сформулированы подходы к дальнейшему построению композитных векторных представлений текстов для улучшения качества классификации.

Основные характеристики разработанного прототипа включают модульную архитектуру пайплайна (предобработка, аугментация, построение эмбеддингов, обучение и оценка моделей), возможность сравнения классических и нейросетевых подходов и готовность к расширению за счёт внедрения композитных эмбеддингов.

Степень внедрения результатов работы определяется возможностью адаптации разработанного прототипа к задачам автоматической классификации корпоративных текстов, в частности маршрутизации обращений и категоризации внутренних документов. Полученные наработки и программная реализация могут быть использованы в качестве основы для последующей интеграции в промышленные системы анализа текстовых данных.

Ключевые слова: МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ, ТЕКСТОВЫЕ ЭМБЕДДИНГИ, КОМПОЗИТНЫЕ ПРЕДСТАВЛЕНИЯ, АУГМЕНТАЦИЯ ТЕКСТОВ, TF-IDF, RUBERT, НЕЙРОСЕТЕВЫЕ МОДЕЛИ.

## СОДЕРЖАНИЕ

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ .....                                                    | 9  |
| ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ .....                                        | 12 |
| ВВЕДЕНИЕ .....                                                                 | 14 |
| 1 Обзор предметной области .....                                               | 17 |
| 1.1 Многоклассовая классификация текстов и её приложения.....                  | 17 |
| 1.2 Особенности классификации длинных русскоязычных деловых писем .....        | 17 |
| 1.3 Векторные представления текста: от Bag-of-Words до контекстных эмбеддингов |    |
| 18                                                                             |    |
| 1.4 Проблемы разреженности, дисбаланса классов и качества текстовых            |    |
| представлений .....                                                            | 19 |
| 1.5 Методы аугментации текстов.....                                            | 21 |
| 1.6 Подходы к обработке длинных текстов в transformer-моделях .....            | 22 |
| 1.7 Идея композитных векторных представлений текста .....                      | 23 |
| 2 Разведочный анализ и предобработка данных .....                              | 24 |
| 2.1 Структура датасета и распределение классов .....                           | 24 |
| 2.2 Анализ длины текстов и качества исходных данных .....                      | 25 |
| 2.3 Обнаружение дубликатов, пересечений и аномальных объектов.....             | 26 |
| 2.4 Правила очистки и нормализации текстов .....                               | 27 |
| 2.5 Изменения в данных после предобработки .....                               | 27 |
| 2.6 Разбиение на train/test .....                                              | 29 |
| 3 Постановка задачи и проектирование решения.....                              | 31 |
| 3.1 Формальная постановка задачи многоклассовой классификации .....            | 31 |
| 3.2 Требования к функциональности и воспроизводимости .....                    | 31 |
| 3.3 Влияние предметной области на проектирование решения .....                 | 32 |
| 3.4 Архитектура пайплайна обработки текста .....                               | 32 |
| 3.5 Выбор направлений моделирования .....                                      | 32 |
| 3.6 Использование суммаризации для подготовки дополнительных представлений     |    |
| текста .....                                                                   | 33 |
| 3.7 Построение суммаризованного и комбинированного наборов данных .....        | 33 |
| 3.8 Выбор метрик качества классификации .....                                  | 34 |

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 4     | Аугментация данных и обогащение выборки.....                                 | 35 |
| 4.1   | Мотивация аугментации при дисбалансе классов .....                           | 35 |
| 4.2   | Аугментация классическими методами без использования нейросетевых моделей    | 35 |
| 4.3   | Общая схема нейросетевой аугментации длинных текстов .....                   | 35 |
| 4.4   | Разбиение длинных текстов для аугментации .....                              | 36 |
| 4.5   | Генерация нескольких вариантов аугментации и выбор итогового кандидата ..... | 36 |
| 4.6   | Схема доведения редких классов до порогового размера выборки.....            | 37 |
| 4.7   | Аугментация с использованием перефразирования .....                          | 38 |
| 4.8   | Аугментация методом обратного перевода.....                                  | 38 |
| 4.9   | Отбор аугментированных текстов по косинусному сходству .....                 | 39 |
| 4.10  | Восстановление полного текста из аугментированных фрагментов .....           | 39 |
| 4.11  | Итоговый аугментированный обучающий набор .....                              | 40 |
| 4.12  | Анализ распределения данных после аугментации.....                           | 40 |
| 5     | Классические модели на основе разреженных текстовых признаков.....           | 42 |
| 5.1   | Теоретические основы TF-IDF-представления текста .....                       | 42 |
| 5.2   | Линейные классификаторы для разреженных текстовых признаков .....            | 42 |
| 5.3   | Признаки на основе словных n-грамм .....                                     | 43 |
| 5.4   | Признаки на основе символьных n-грамм.....                                   | 43 |
| 5.5   | Объединение словных и символьных признаков.....                              | 44 |
| 5.6   | Реализация пайплайна TF-IDF + LinearSVC .....                                | 44 |
| 5.7   | Реализация пайплайна TF-IDF + LogisticRegression.....                        | 45 |
| 5.8   | Особенности TF-IDF для длинных деловых писем .....                           | 45 |
| 6     | Контекстные представления и классификация на основе BERT-моделей .....       | 47 |
| 6.1   | Теоретические основы transformer-архитектур.....                             | 47 |
| 6.2   | Контекстные эмбединги и их отличие от разреженных признаков .....            | 47 |
| 6.3   | Используемые русскоязычные модели .....                                      | 48 |
| 6.4   | Два подхода к использованию BERT в задаче классификации.....                 | 49 |
| 6.4.1 | Дообучение Sentence-BERT для построения эмбедингов.....                      | 49 |
| 6.4.2 | Дообучение BERT как сквозного классификатора.....                            | 51 |
| 6.5   | Классификация через специальный токен CLS .....                              | 51 |
| 6.6   | Классификация через усреднение скрытых состояний токенов (mean pooling)....  | 52 |

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 6.7 Эксперименты с ruBERT-base-cased и различными классификационными головами | 52 |
| 6.8 Эксперименты с ruRoberta-large и различными классификационными головами   | 53 |
| 6.9 Ограничения длины входной последовательности в BERT-моделях               | 54 |
| 6.10 Chunked-классификатор: архитектура для длинных документов                | 54 |
| 6.11 Классификационная голова и функция потерь                                | 55 |
| 6.12 Параметры обучения chunked-классификатора                                | 56 |
| 6.13 Приёмы регуляризации и стабилизации обучения                             | 58 |
| 7 Композитные и эмбединговые представления текста                             | 60 |
| 7.1 Варианты BERT-эмбедингов: предобученные и дообученные                     | 60 |
| 7.2 Классические классификаторы поверх плотных эмбедингов                     | 60 |
| 7.3 Методы классификации по косинусному сходству                              | 61 |
| 7.4 Конкатенация TF-IDF и BERT-векторов                                       | 63 |
| 7.5 Реализация композитного признакового пространства                         | 64 |
| 7.6 Классические модели и MLP поверх композитных векторов                     | 65 |
| 7.7 Роль композитных признаков в общей схеме                                  | 66 |
| 8 Программная реализация пайплайна                                            | 68 |
| 8.1 Структура программного решения                                            | 68 |
| 8.2 Модуль предобработки данных                                               | 69 |
| 8.3 Модуль аугментации текстов                                                | 69 |
| 8.4 Модуль построения векторных представлений                                 | 70 |
| 8.5 Модуль обучения моделей                                                   | 71 |
| 8.6 Воспроизводимость экспериментов                                           | 71 |
| 8.7 Ограничения реализации и требования к вычислительным ресурсам             | 72 |
| 9 Экспериментальное исследование и сравнение результатов                      | 74 |
| 9.1 Экспериментальный протокол оценки качества                                | 74 |
| 9.2 Используемые метрики качества классификации                               | 74 |
| 9.3 Влияние аугментации на TF-IDF и BERT-модели                               | 75 |
| 9.4 Влияние суммаризации на представление текста                              | 76 |
| 9.5 Сравнение классических, нейросетевых и композитных представлений          | 80 |
| 9.6 Сводная таблица результатов лучших реализованных подходов                 | 83 |
| 9.7 Выводы по результатам экспериментального исследования                     | 85 |

|                                        |    |
|----------------------------------------|----|
| ЗАКЛЮЧЕНИЕ.....                        | 87 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... | 89 |



## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяют следующие термины с соответствующими определениями:

|                                                   |   |                                                                                                                                                         |
|---------------------------------------------------|---|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Многоклассовая классификация текста               | — | задача автоматического отнесения текстового документа к одному из заранее заданных классов из конечного множества категорий                             |
| Векторное представление текста (эмбеддинг текста) | — | численное представление текста в виде вектора признаков, используемое моделью машинного обучения для классификации или сравнения документов.            |
| Композитное векторное представление текста        | — | признаковое представление документа, полученное путём объединения нескольких типов признаков, например TF-IDF-вектора и BERT-эмбеддинга                 |
| Текстовая аугментация                             | — | совокупность методов увеличения обучающей выборки за счёт генерации дополнительных текстов на основе исходных документов с сохранением их целевой метки |
| Перефразирование текста                           | — | метод текстовой аугментации, при котором исходный текст преобразуется в близкий по смыслу вариант с изменённой лексической или синтаксической формой    |
| Обратный перевод                                  | — | метод текстовой аугментации, при котором текст переводится на промежуточный язык, а затем обратно на исходный язык для получения новой формулировки     |

|                                          |                                                                                                                                                               |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Базовая модель классификации             | — модель, реализующая простой или широко используемый подход и применяемая как отправная точка для сравнения с более сложными решениями                       |
| Нейросетевая модель классификации текста | — модель классификации, основанная на нейронных сетях и использующая плотные или контекстные представления текста                                             |
| Корпоративный датасет текстов            | — набор текстовых данных, сформированный на основе документов или переписки организации и отражающий реальные процессы предметной области                     |
| Деловое письмо                           | — текстовый документ корпоративной переписки, содержащий служебную, управленческую, проектную или организационную информацию                                  |
| Разреженное представление текста         | — векторное представление, в котором большинство значений равно нулю; в работе к таким представлениям относятся TF-IDF-векторы на основе word- и char n-грамм |
| Контекстное эмбединговое представление   | — плотное векторное представление текста, полученное с помощью языковой модели и учитывающее контекст употребления слов                                       |
| Суммаризация текста                      | — метод построения сокращённого представления документа, сохраняющего его основное содержание                                                                 |
| Чанк                                     | — фрагмент длинного текста, полученный при разбиении документа на части ограниченной длины для обработки BERT-подобной моделью                                |
| Классификация по центроидам              | — метод классификации в эмбединговом пространстве, при котором для каждого класса вычисляется средний вектор, а новый объект                                  |

|                                                       |                                                                                                                                                                                                        |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Классификация</p> <p>методом ближайшего соседа</p> | <p>относится к классу с наиболее близким центроидом</p> <p>— метод классификации, при котором метка нового документа определяется по наиболее близкому обучающему примеру в векторном пространстве</p> |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяют следующие сокращения и обозначения:

|                    |   |                                                                                                                                                                 |
|--------------------|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BERT               | — | Bidirectional Encoder Representations from Transformers, языковая модель на основе encoder-части transformer-архитектуры                                        |
| ruBERT             | — | русскоязычная BERT-модель, предварительно обученная на русскоязычных текстах                                                                                    |
| RoBERTa            | — | модификация BERT, использующая изменённую стратегию предварительного обучения                                                                                   |
| ruRoberta-large    | — | русскоязычная RoBERTa-модель большого размера, используемая в работе для классификации и построения эмбеддингов                                                 |
| SBERT              | — | Sentence-BERT, подход к построению эмбеддингов предложений и документов на основе BERT-подобных моделей                                                         |
| TF-IDF             | — | Term Frequency - Inverse Document Frequency, метод взвешенного представления текста, учитывающий частоту термина в документе и его распространённость в корпусе |
| SVM                | — | Support Vector Machine, метод опорных векторов                                                                                                                  |
| LinearSVC          | — | линейный классификатор на основе метода опорных векторов, используемый для работы с разреженными признаками                                                     |
| LogisticRegression | — | логистическая регрессия, линейная модель классификации                                                                                                          |
| MLP                | — | Multi-Layer Perceptron, многослойный перцептрон                                                                                                                 |
| NLP                | — | Natural Language Processing, обработка естественного языка                                                                                                      |

|                   |   |                                                                                                                                      |
|-------------------|---|--------------------------------------------------------------------------------------------------------------------------------------|
| F1                | — | гармоническое среднее между точностью (precision) и полнотой (recall), используемое для оценки качества классификации                |
| Macro F1          | — | макро-усреднённая F1-мера, при которой F1 рассчитывается отдельно для каждого класса, после чего значения усредняются с равным весом |
| Accuracy          | — | доля правильно классифицированных объектов в общей выборке                                                                           |
| Balanced accuracy | — | сбалансированная точность, рассчитываемая как среднее значение полноты по классам                                                    |
| GPU               | — | Graphics Processing Unit, графический процессор, используемый для ускорения вычислений при обучении нейросетевых моделей             |
| EDA               | — | Exploratory Data Analysis, первичный исследовательский анализ данных                                                                 |

## ВВЕДЕНИЕ

С каждым годом степень цифровизации в профессиональной и повседневной жизни только увеличивается. Это мотивирует внедрение максимальной автоматизации, не только как улучшение, а как единственно возможный способ обработки всего входящего потока информации. Одним из таких направлений является классификация текстов. Современные методы машинного обучения открывают широкие возможности, однако на практике они еще не получили повсеместного распространения. В данной работе рассматривается задача классификации русскоязычных длинных деловых писем организации, включающей 36 отделов. Целью является разработка решения, которое может быть использовано для автоматической маршрутизации входящих писем в соответствующие подразделения.

В изначальных данных часть классов содержит значительное число документов, тогда как другие представлены ограниченным числом примеров. Такая структура типична для корпоративной переписки, где одни направления деятельности создают постоянный поток документов, а другие встречаются значительно реже. В этих условиях качество классификации зависит не только от выбранной модели, но и от подготовки данных, выбора метрик, а также способов расширения обучающей выборки.

Традиционные подходы к представлению текста, основанные на TF-IDF, остаются важной базовой линией для задач классификации. Они позволяют эффективно использовать линейные классификаторы, быстро обучаются и хорошо фиксируют характерные слова, n-граммы, аббревиатуры и устойчивые деловые формулировки. Одновременно такие подходы ограниченно учитывают контекст и смысловую близость разных формулировок. Поэтому в работе дополнительно рассматриваются BERT-подобные модели, способные строить контекстные представления текста.

В рамках исследования реализуются несколько направлений моделирования. Классическое направление основано на TF-IDF-признаках и линейных классификаторах. Нейросетевое направление включает ruBERT и ruRoberta-large, в том числе варианты классификации через стандартную голову, mean pooling и отдельную архитектуру для длинных документов с разбиением на чанки. Также рассматриваются эмбединговые подходы, Sentence-BERT-представления, классификация по косинусному сходству, суммаризация документов и композитные признаки, объединяющие TF-IDF и BERT-векторы.

Актуальность работы определяется необходимостью построения воспроизводимого пайплайна для классификации длинных русскоязычных деловых писем в условиях

ограниченной и несбалансированной обучающей выборки. Такая задача требует не только выбора модели, но и комплексной организации обработки данных: очистки, анализа распределения классов, аугментации, суммаризации, построения разных признаковых пространств и корректного сравнения результатов.

Цель работы заключается в разработке и исследовании пайплайна многоклассовой классификации длинных русскоязычных деловых писем на основе классических, нейросетевых, эмбединговых и композитных представлений текста.

Для достижения поставленной цели в работе решаются следующие задачи:

- 1) провести разведочный анализ исходного датасета, изучить распределение классов, длину документов и качество текстов;
- 2) разработать и реализовать процедуры очистки и нормализации текстов с учётом специфики деловой переписки;
- 3) сформировать фиксированное train/test-разбиение и обеспечить отсутствие использования тестовых данных при обучении и аугментации; построить базовый классический пайплайн классификации на основе TF-IDF и линейного классификатора и оценить его качество;
- 4) реализовать методы обогащения обучающей выборки, включая классическую обработку, перефразирование и обратный перевод;
- 5) построить дополнительные варианты текстовых представлений на основе суммаризации и комбинирования исходного текста с summary;
- 6) реализовать классические модели классификации на основе TF-IDF-признаков, включая word-, char- и объединённые word+char-представления;
- 7) реализовать и исследовать BERT-подходы на основе ruBERT и ruRoberta-large, включая варианты с различными классификационными головами и чанкингом длинных документов;
- 8) построить BERT- и Sentence-BERT-эмбединги документов и исследовать классификацию на их основе;
- 9) реализовать композитное признаковое пространство, объединяющее TF-IDF и BERT-векторы;
- 10) провести экспериментальное сравнение реализованных подходов по метрикам balanced accuracy и macro F1.

Объектом исследования является процесс автоматической многоклассовой классификации русскоязычных деловых писем.

Предметом исследования являются методы предобработки, аугментации, суммаризации и построения векторных представлений текста, применяемые для классификации длинных документов в условиях дисбаланса классов.

Теоретическая значимость работы состоит в систематизации и сравнении нескольких подходов к представлению длинных текстов: разреженных TF-IDF-признаков, контекстных BERT-представлений, Sentence-BERT-эмбеддингов и композитных векторов. В работе рассматривается влияние не только модели, но и всего пайплайна обработки данных на итоговое качество классификации.

Практическая значимость работы заключается в разработке модульного программного прототипа, включающего предобработку данных, аугментацию, суммаризацию, построение признаков, обучение моделей и сравнение результатов. Предложенный пайплайн может быть адаптирован для задач автоматической маршрутизации корпоративных писем, категоризации внутренних документов и поддержки систем электронного документооборота.



## **1 Обзор предметной области**

### **1.1 Многоклассовая классификация текстов и её приложения**

Многоклассовая классификация текстов — это задача автоматического отнесения документа к одной из нескольких заранее заданных категорий. Такая задача возникает в тех случаях, когда текстовый поток необходимо распределить по конкретным темам.

На практике многоклассовая классификация используется в системах электронного документооборота, службах поддержки, корпоративных почтовых системах, новостных агрегаторах, юридических и финансовых информационных системах. Например, входящее письмо может быть автоматически отнесено к определённому проекту, отделу, типу запроса или зоне ответственности сотрудника. Это позволяет сократить ручную обработку документов.

В рамках данной работы рассматривается прикладной вариант этой задачи: классификация длинных русскоязычных деловых писем по 36 классам. Документы имеют значительную длину, содержат формальные обороты, подписи, пересылаемые фрагменты, служебные конструкции и предметно-специфичную лексику. Кроме того, отдельные классы могут быть близки по смыслу, так как относятся к смежным подразделениям, проектам или направлениям деятельности. Поэтому модель должна учитывать не только отдельные ключевые слова, но и общий контекст письма.

Дополнительную сложность создаёт ограниченный и несбалансированный набор данных. В исходной обучающей выборке разные классы представлены неравномерно: одни категории содержат большое количество писем, а другие имеют только несколько примеров. В такой ситуации модель может смещаться в сторону крупных классов и хуже распознавать редкие категории. Поэтому в работе задача классификации рассматривается совместно с анализом распределения классов, предобработкой текстов, аугментацией данных и сравнением разных способов построения текстовых представлений.

### **1.2 Особенности классификации длинных русскоязычных деловых писем**

В деловой переписке текст часто строится по более сложной структуре: в одном письме могут присутствовать приветствие, описание ситуации, ссылки на предыдущие договорённости, пересылаемые фрагменты, подписи, реквизиты, уточнения по срокам и исполнителям. При этом не все части письма одинаково полезны для определения класса.

Например, подписи и служебные блоки могут повторяться в разных категориях, а ключевая информация может находиться только в одном или нескольких абзацах.

В рассматриваемой задаче документы являются длинными русскоязычными деловыми письмами. Это накладывает несколько ограничений на выбор методов обработки текста. Во-первых, длинные документы сложнее подавать на вход transformer-моделям, так как большинство BERT-подобных архитектур имеет ограничение на длину входной последовательности. Если просто обрезать письмо до фиксированного числа токенов, часть значимой информации может быть потеряна. Во-вторых, длинный текст часто содержит информационный шум: повторяющиеся служебные конструкции, фрагменты переписки, стандартные формулировки и элементы, которые не помогают отличать один класс от другого.

Русский язык дополнительно усложняет задачу за счёт развитой морфологии и свободного порядка слов. Одни и те же сущности, действия и признаки могут встречаться в разных грамматических формах. Для классических моделей это означает увеличение числа возможных токенов и разреженность признакового пространства. Для нейросетевых моделей проблема проявляется иначе: модель может лучше учитывать контекст, но при малом количестве размеченных примеров не всегда получает достаточно данных для устойчивого дообучения под конкретную предметную область.

Специфика деловой переписки также связана с высокой тематической близостью классов. В отличие от задач, где категории хорошо различимы по лексике, в корпоративных письмах разные классы могут использовать сходные термины, шаблоны и управленческие формулировки. Например, письма, относящиеся к разным подразделениям или проектам, могут содержать одинаковые слова о согласовании, сроках, документах, исполнителях и отчётности. Поэтому классификатору необходимо выделять не только общеделовую лексику, но и более тонкие признаки, связанные с конкретным направлением или контекстом письма.

### **1.3 Векторные представления текста: от Bag-of-Words до контекстных эмбеддингов**

Для применения алгоритмов машинного обучения текст должен быть преобразован в численное представление — вектор признаков фиксированной длины. Самые ранние подходы опирались на модели one-hot и Bag-of-Words, в которых каждый текст описывается вектором, отражающим частоты или наличие слов из словаря. Расширение этой идеи позволяет взвешивать слова с учётом их частоты в документе и редкости в

коллекции, снижая влияние часто встречающихся, но малоинформативных терминов и наоборот повышая редкие, но высокоинформативные термины — TF-IDF.

Но такие представления не учитывают порядок слов и семантическую близость, а также страдают от разреженности при больших словарях. Для преодоления этих ограничений были предложены плотные распределённые представления. Такими методами стали Word2Vec, GloVe и FastText, в которых каждая лексема отображается в вектор невысокой размерности, обученный на больших корпусах. Революционным шагом стало появление контекстных эмбеддингов, в том числе на основе архитектуры BERT, которые формируют представление токена или целого текста с учётом контекста. Это позволяет различать многозначные слова и тонкие смысловые нюансы.

Поскольку разные типы представлений фиксируют разные свойства текста, в работе также рассматривается идея композитного признакового пространства. В таком подходе разреженные TF-IDF-признаки объединяются с плотными BERT-векторами, после чего поверх общего представления обучается классификатор. Цель такого подхода заключается в том, чтобы совместить сильные стороны классических и нейросетевых представлений: чувствительность TF-IDF к характерным словам и фразам, а также способность BERT-моделей учитывать контекст и семантическую близость текстов.

Из-за перечисленных особенностей в работе рассматривается несколько способов представления и обработки текстов. Для классического подхода используются TF-IDF-признаки, включая word и char n-граммные представления, так как они позволяют учитывать как отдельные слова, так и устойчивые фрагменты словоформ. Для нейросетевых моделей исследуются BERT-модели, которые способны строить контекстные представления текста. Отдельно рассматриваются композитные представления, объединяющие разные типы признаков.

#### **1.4 Проблемы разреженности, дисбаланса классов и качества текстовых представлений**

При классификации текстов качество модели во многом зависит от того, насколько удачно исходные документы преобразованы в признаковое пространство. В классических подходах, таких как Bag-of-Words и TF-IDF, каждый документ описывается большим количеством признаков, соответствующих словам, n-граммам или символьным фрагментам. Такое пространство обычно получается высокоразмерным и разреженным: большая часть признаков для конкретного документа имеет нулевые значения. Это

естественно для текстовых данных, так как даже длинное письмо содержит только малую часть всех возможных слов и сочетаний, встречающихся в корпусе.

Разреженность сама по себе не делает метод непригодным. Линейные модели часто хорошо работают с такими признаками, поскольку могут выделять информативные слова и устойчивые выражения для каждого класса. Однако при малом количестве обучающих примеров возникают ограничения. Если редкий класс представлен несколькими письмами, модель видит слишком мало вариантов формулировок и может переобучиться на случайные слова, подписи, имена проектов или служебные фрагменты. В результате признаки, которые не отражают сущность класса, могут получить слишком большой вес.

В рассматриваемой задаче эта проблема усиливается дисбалансом классов. Исходная обучающая выборка содержит 36 классов, но количество документов по ним распределено неравномерно: для части классов доступно большое число примеров, а отдельные категории представлены единичными письмами. В такой ситуации модель может показывать приемлемое общее качество за счёт крупных классов, но плохо распознавать малые классы. Поэтому при анализе результатов важно использовать не только общую долю правильных ответов, но и метрики, чувствительные к качеству на каждом классе, например *balanced accuracy* и *macro F1*.

Дисбаланс влияет не только на классические модели, но и на нейросетевые подходы. BERT-подобные модели обладают более выразительными представлениями, однако для дообучения под конкретную предметную область им также требуется достаточное количество размеченных примеров. Если данных мало, а классы близки по тематике, модель может не получить устойчивого сигнала для разделения категорий. Особенно это заметно в задачах с длинными документами, где значимая информация может занимать небольшую часть текста, а остальное содержимое создаёт шум.

Качество признаков зависит от того, какие сигналы из текста модель вообще «видит». Словные признаки хорошо ловят термины и устойчивые фразы, но легко теряются, когда одна и та же мысль выражена другими словами. Символьные *n*-граммы, наоборот, помогают с морфологией, сокращениями и частичными совпадениями. Контекстные BERT-эмбединги лучше передают смысл, но для длинных писем упираются в ограничение длины входа, а при небольшом числе примеров их сложнее надёжно адаптировать под предметную область. Поэтому в работе сравниваются несколько вариантов представления текста: TF-IDF, BERT-эмбединги и их композитные комбинации. Чтобы снизить влияние дисбаланса и расширить набор формулировок,

аугментация применяется только к обучающей, в первую очередь для редких классов. Дополнительно проверяются суммаризованные и комбинированные варианты, чтобы понять, можно ли сжать письмо без потери признаков, важных для классификации.

## **1.5 Методы аугментации текстов**

Аугментация текстов применяется в тех случаях, когда обучающая выборка слишком мала или заметно несбалансирована по классам. Её задача — получить дополнительные примеры на основе уже размеченных данных, сохраняя исходную метку, но меняя формулировки или структуру текста. Это помогает увеличить разнообразие обучающих примеров и снижает риск того, что модель будет запоминать отдельные случайные фрагменты вместо общего смысла. При этом важно соблюдать баланс: слишком сильные изменения могут исказить содержание текста, поэтому особенно для редких классов необходимо генерировать такие варианты, которые сохраняют исходный смысл и корректную принадлежность к классу.

Существуют и простые способы аугментации, не требующие нейросетевых моделей. К ним относятся, например, замена слов синонимами, удаление или перестановка отдельных слов, а также небольшие изменения пунктуации. Такие методы практически не требуют вычислительных ресурсов. Однако для длинных деловых писем их применение ограничено: в таких текстах важны точные формулировки, терминология и стиль, поэтому случайные изменения часто приводят не к полезному разнообразию, а к появлению шума в данных.

Более содержательными являются методы нейросетевой аугментации. В данной работе рассматриваются два таких подхода: перефразирование и обратный перевод. Перефразирование направлено на получение нового варианта текста с сохранением исходного смысла. Обратный перевод строится иначе: текст переводится на промежуточный язык, а затем обратно на русский. В результате часть формулировок изменяется, но общий смысл документа должен сохраняться. Оба метода позволяют получить более естественные варианты текста по сравнению с простыми случайными заменами.

Для длинных писем аугментация полного текста целиком может быть затруднена из-за ограничений моделей и высокой длины входной последовательности. Поэтому в работе используется обработка на уровне фрагментов: документ разбивается на предложения и более короткие части, для которых генерируются варианты аугментации. После этого

фрагменты объединяются обратно в полный текст. Такой подход позволяет применять модели аугментации к длинным документам и при этом сохранять общую структуру письма.

Важным этапом является контроль качества сгенерированных текстов. Если аугментированный пример слишком близок к исходному, он почти не добавляет новой информации для обучения. Если же он слишком сильно отличается, возникает риск изменить смысл и получить пример с некорректной меткой. Поэтому в работе используется фильтрация по косинусному сходству между исходным и аугментированным текстом. В итоговую обучающую выборку включаются только те варианты, которые сохраняют достаточную смысловую близость к оригиналу, но не являются его почти точной копией.

## **1.6 Подходы к обработке длинных текстов в transformer-моделях**

При работе с длинными документами transformer-модели имеют важное ограничение: стандартные BERT-подобные архитектуры принимают на вход последовательность ограниченной длины. На практике это означает, что длинное письмо не всегда может быть обработано целиком. Если документ просто обрезается до максимально допустимой длины, часть информации может быть потеряна, причём заранее неизвестно, где именно находится фрагмент, важный для классификации.

Один из практических способов работы с длинными текстами заключается в разбиении документа на несколько фрагментов. Каждый фрагмент подаётся в transformer-модель отдельно, после чего полученные представления объединяются на уровне документа. Фрагменты могут формироваться с перекрытием, чтобы информация на границах частей не терялась. Такой подход позволяет использовать стандартные BERT-подобные модели без изменения их архитектуры и при этом учитывать большую часть исходного письма.

В данной работе этот принцип используется при реализации классификатора на основе ruRoberta-large для длинных документов. Текст разбивается на чанки фиксированной длины с заданным шагом, после чего модель обрабатывает несколько фрагментов одного письма. Далее признаки фрагментов агрегируются и передаются в классификационную голову. Такой подход выбран как компромисс между полнотой использования текста и вычислительными ограничениями, поскольку обработка всех возможных фрагментов длинного письма может быть слишком затратной.

Помимо чанкинга, для длинных текстов может применяться суммаризация. В этом случае из исходного документа формируется сокращённая версия, которая должна сохранять основное содержание. Такой вариант полезен для проверки гипотезы о том, достаточно ли краткого представления письма для определения класса. В работе поэтому рассматриваются не только исходные тексты, но и суммаризованные документы, а также комбинированный вариант, где суммаризированный текст используется вместе с оригиналом. Это позволяет сравнить, какую роль играет полный текст и насколько информативным оказывается его сжатое представление.

## **1.7 Идея композитных векторных представлений текста**

Разные способы представления текста фиксируют разные свойства документа. Разреженные признаки, построенные с помощью TF-IDF, хорошо отражают наличие характерных слов, устойчивых словосочетаний и символьных фрагментов. Они особенно полезны в тех случаях, когда класс можно частично определить по специфичной терминологии, названиям проектов, подразделений, типовым формулировкам или повторяющимся деловым оборотам. При этом такие признаки слабо учитывают общий смысл текста и не всегда устойчивы к изменению формулировок.

Контекстные представления, которые дают BERT-подобные модели, хорошо улавливают окружение слов и смысл текста в целом. За счёт этого разные формулировки одного и того же обращения могут иметь близкие векторы, даже если слова отличаются. Для деловой переписки это важно, потому что один тип запроса сотрудники формулируют по-разному. При этом плотные эмбединги не всегда явно подсвечивают редкие, но значимые токены, которые могут быть критичны для отнесения письма к конкретному классу.

Идея композитных векторных представлений состоит в том, чтобы объединить сильные стороны разных типов признаков в одном векторе. В работе это реализуется через конкатенацию разреженного TF-IDF-представления и плотного BERT-вектора. TF-IDF часть отвечает за точные совпадения слов, n-грамм и характерных текстовых шаблонов, а BERT-компонента добавляет информацию о семантике и контексте. Поверх такого объединённого пространства обучаются стандартные классификаторы — LinearSVC, LogisticRegression и MLP, что позволяет проверить, даёт ли совместное использование разреженных и плотных признаков выигрыш по сравнению с каждым из них по отдельности.

## 2 Разведочный анализ и предобработка данных

### 2.1 Структура датасета и распределение классов

В работе используется набор русскоязычных деловых писем, размеченных по тематическим классам. Каждый объект датасета представляет собой отдельный текстовый документ, которому сопоставлена одна целевая метка. Конфиденциальная информация в документах заменена масками-заполнителями. Метка соответствует направлению, проекту, подразделению или иной категории, используемой для классификации письма. Таким образом, задача рассматривается как классификация одного документа в один из заранее заданных классов.

Всего в наборе данных представлено 36 классов. Такое количество категорий делает задачу более сложной по сравнению с бинарной классификацией или классификацией на небольшое число тем. При этом классы не являются равномерными по объёму: часть категорий содержит большое число документов, а некоторые представлены существенно меньшим количеством примеров, распределение представлено на рисунке 2.1. Для реальных деловых данных такая ситуация является естественной, так как интенсивность переписки по разным направлениям может значительно отличаться.

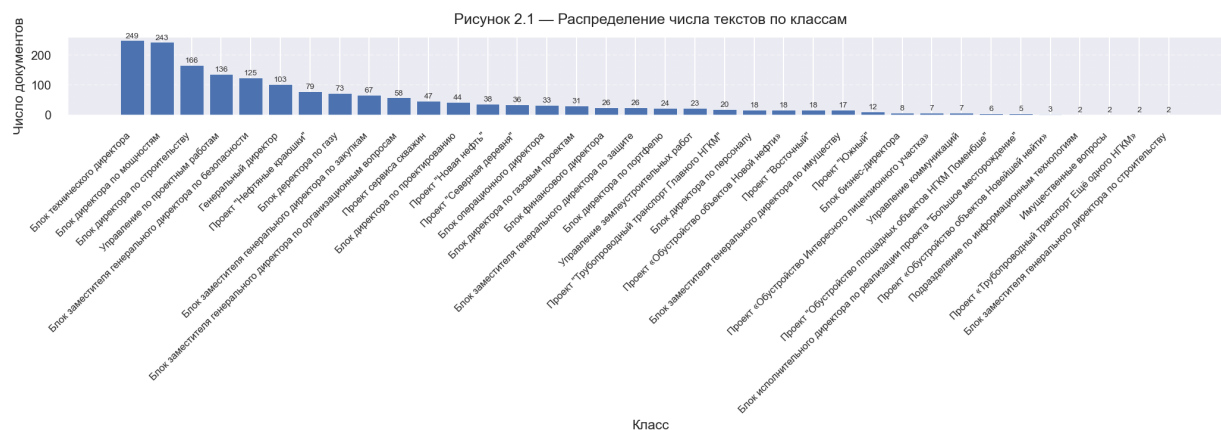


Рисунок 2.1 — Распределение числа текстов по классам

Анализ распределения классов показывает, что датасет имеет выраженный дисбаланс. Наиболее крупные классы содержат существенно больше документов, чем редкие категории. Это означает, что разные классы представлены в обучающих данных с разной степенью полноты: для крупных категорий модель может наблюдать больше вариантов формулировок, а для малых — только ограниченное число примеров. Данная



особенность учитывается при выборе метрик качества и при проектировании этапа аугментации.

## 2.2 Анализ длины текстов и качества исходных данных

В рамках анализа были рассмотрены основные характеристики длины документов: количество символов, слов и приблизительное число токенов. Соответствующие распределения показаны на рисунках 2.2 и 2.3. Для моделей на основе TF-IDF длина текста сама по себе не является ограничением, но влияет на размер признакового пространства и может усиливать вклад повторяющихся служебных фрагментов. Для BERT-моделей длина играет более важную роль, так как входная последовательность ограничена фиксированным числом токенов.

Рисунок 2.2 — Распределение длины документов по числу символов

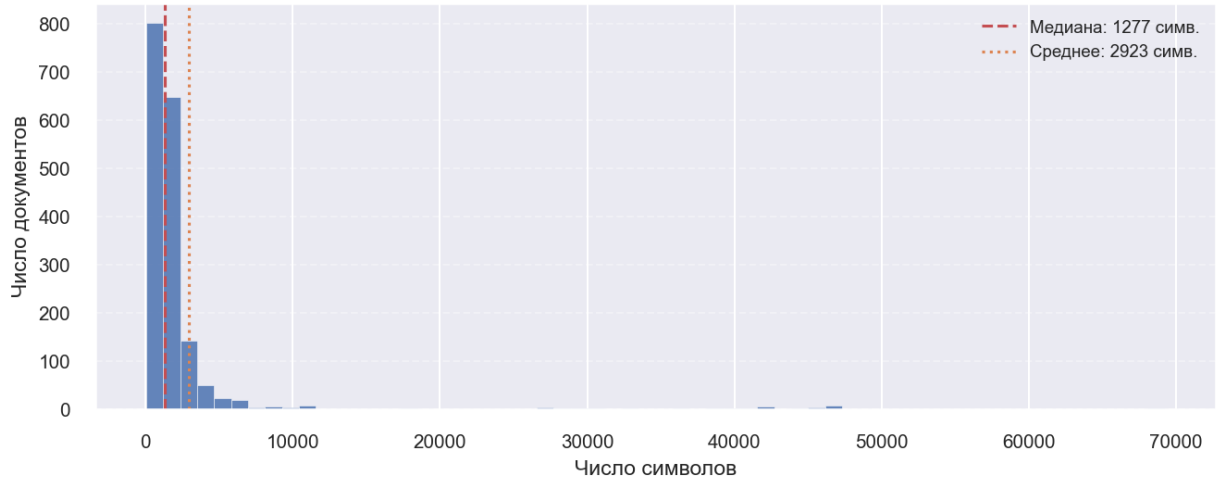


Рисунок 2.2 — Распределение количества символов в документах

Рисунок 2.3 — Распределение длины документов (до очистки)

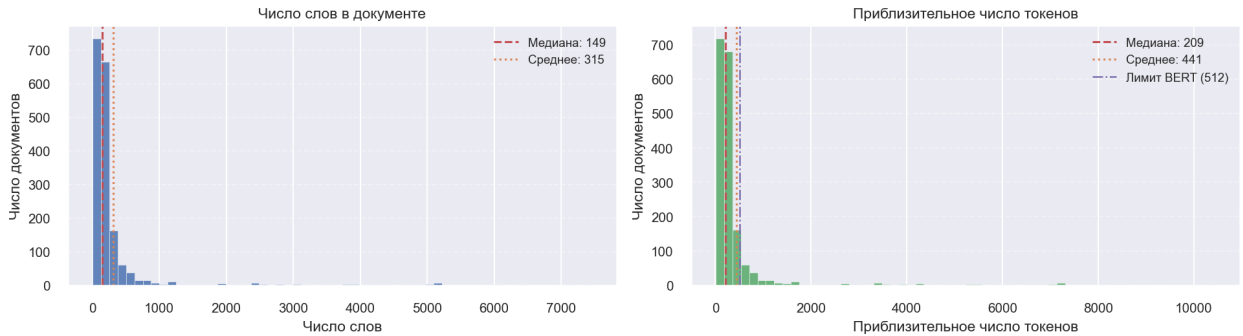


Рисунок 2.3 — Распределение числа слов и токенов в документах

Анализ показал, что в датасете присутствуют как короткие, так и достаточно длинные письма. Помимо длины, отдельно рассматривалось качество исходных данных. В деловой переписке часто встречаются элементы, которые не несут прямой пользы для классификации: подписи, приветствия, повторяющиеся блоки, служебные символы, ссылки, шаблонные формулировки. Такие элементы могут встречаться в разных классах и создавать шум в признаковом пространстве. При этом полностью удалять все служебные конструкции нельзя, поскольку часть из них может быть связана с конкретным подразделением, проектом или типом документа.

В результате анализа было установлено, что исходный набор данных требует предварительной очистки и нормализации, но при этом содержит достаточно информативные тексты для построения классификационного пайплайна. Основные ограничения связаны с неоднородной длиной документов, наличием служебных фрагментов и потенциальной потерей информации при обработке длинных писем transformer-моделями. Эти факторы были учтены при проектировании этапов предобработки, аугментации, суммаризации и классификации.

### **2.3 Обнаружение дубликатов, пересечений и аномальных объектов**

На этапе подготовки данных была выполнена проверка набора писем на наличие дубликатов, пересечений между выборками и аномальных объектов. Для задачи классификации это является важным этапом, поскольку качество итоговой модели должно оцениваться на документах, которые не использовались при обучении. Если одинаковые или почти одинаковые тексты оказываются одновременно в обучающей и тестовой выборках, итоговые метрики могут быть завышены и не будут отражать реальную способность модели классифицировать новые письма.

В первую очередь были проанализированы точные дубликаты текстов. Под точным дубликатом в данном случае понимается полное совпадение содержимого письма после базовой нормализации. Такие объекты могут появляться из-за повторной выгрузки данных, технического объединения нескольких файлов или повторного сохранения одного и того же письма. Наличие точных дубликатов внутри обучающей выборки не всегда приводит к утечке данных, но может усиливать вклад отдельных документов и искажать распределение классов. Поэтому такие случаи необходимо учитывать при дальнейшей обработке.

Кроме дубликатов, анализировались аномальные объекты. К ним относились письма с очень малым количеством содержательного текста, документы с чрезмерно большой длиной, а также тексты, состоящие преимущественно из технических элементов. Такие объекты могут по-разному влиять на модели. Слишком короткие письма содержат мало признаков для классификации, а слишком длинные могут включать пересланные цепочки, подписи, повторяющиеся блоки и служебные строки, которые создают шум.

При обработке аномальных объектов использовался осторожный подход. Полное удаление всех нестандартных фрагментов может привести к потере полезной информации, так как в деловых письмах значимыми могут быть названия подразделений, должности, номера проектов, аббревиатуры и служебные обозначения. Поэтому основной целью являлось не агрессивное сокращение текста, а устранение явно технических проблем и проверка того, что данные пригодны для дальнейшего обучения моделей.

## **2.4 Правила очистки и нормализации текстов**

После анализа структуры датасета и примеров писем был сформирован набор правил очистки и нормализации текстов. Их цель — убрать технический шум и привести документы к более однородному виду, не теряя при этом признаков, важных для классификации. В очистку входили приведение регистра, нормализация пробелов, обработка технических фрагментов и унификация типовых элементов, а в отдельных случаях — замена конкретных значений (ссылок, e-mail-адресов и т.п.) на специальные токены, чтобы сохранить сам факт их присутствия, но не привязывать модель к конкретному значению. При этом учитывалось, что TF-IDF выигрывает от более агрессивной очистки за счёт сокращения словаря и уменьшения числа случайных n-грамм, тогда как для BERT-подобных моделей чрезмерная предобработка может быть вредна, поскольку нарушает естественную структуру текста. Поэтому финальные правила подобраны так, чтобы снизить уровень шума, но сохранить связность письма и естественный вид делового текста.

## **2.5 Изменения в данных после предобработки**

После применения правил очистки количество объектов сократилось с 1774 до 1755, при этом число классов осталось прежним — 36. Удалённые записи относились к технически некорректным или недостаточно содержательным письмам и не приводили к исчезновению каких-либо классов из датасета. Гораздо сильнее изменился общий объём

текста: суммарный размер корпуса уменьшился с 5 186 469 до 2 173 226 символов, то есть была удалена более половины символов, главным образом за счёт служебных блоков, повторяющихся фрагментов и технических элементов, создававших шум для моделей классификации. Обновлённые распределения длины документов показаны на рисунках 2.4 и 2.5.

При этом предобработка выполнялась без агрессивного стемминга и лемматизации, чтобы не разрушить доменные термины и названия подразделений. Сохранялись ключевые аббревиатуры и буквенно-цифровые обозначения, которые могут влиять на принадлежность документа к классу. Такой подход позволил уменьшить шум и сократить объём текста без потери важной информации.

Рисунок 2.4 — Распределение длины документов по числу символов (после очистки)

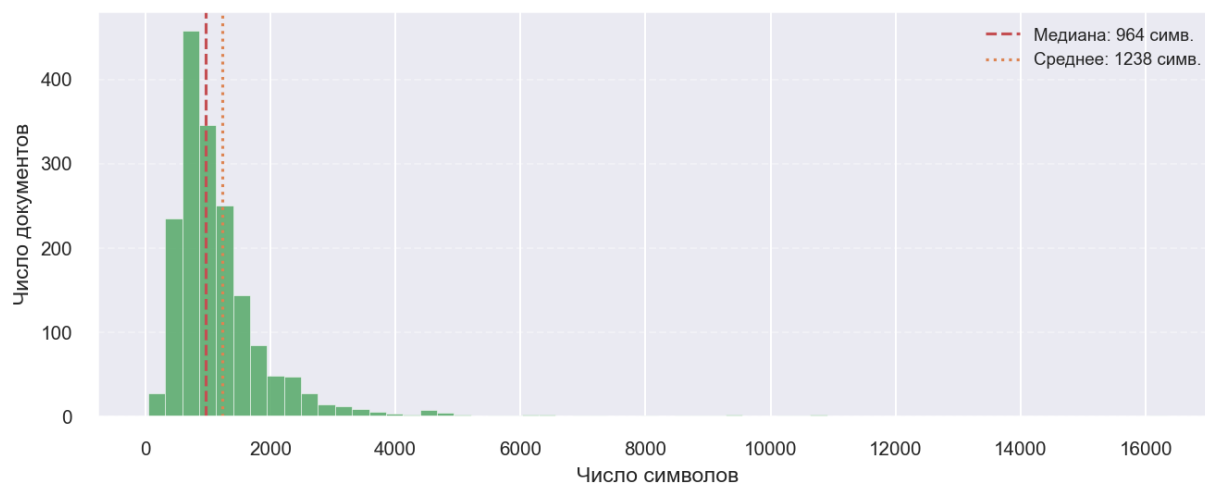


Рисунок 2.4 — Распределение числа символов в документах после очистки

Рисунок 2.5 — Распределение длины документов (после очистки)

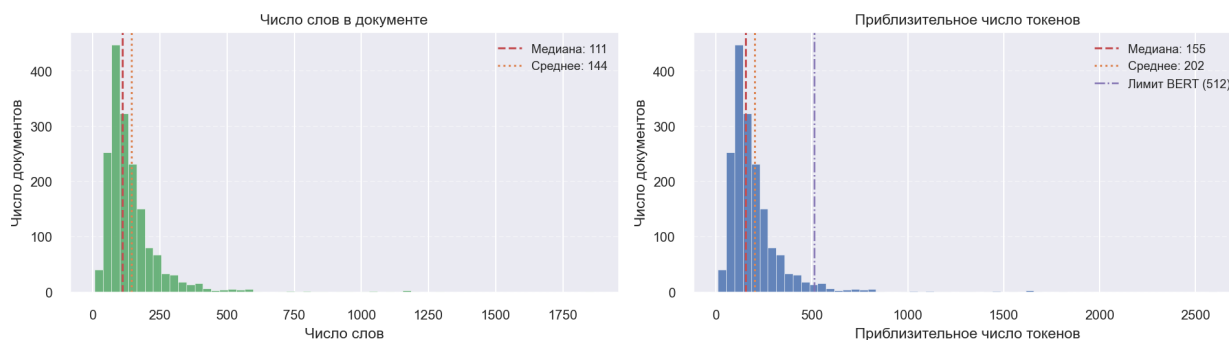


Рисунок 2.5 — Распределение числа слов и токенов в документах после очистки

## 2.6 Разбиение на train/test

После очистки и нормализации был сформирован итоговый корпус, который разделили на обучающую и тестовую выборки. Обучающая часть использовалась для подготовки признаков, обучения моделей и всех дополнительных преобразований данных, тогда как тестовая выборка оставалась неизменной и применялась только для финальной оценки качества классификации.

Таблица 2.1 — Структура train.csv и test.csv

| Часть выборки | Количество документов | Количество классов |
|---------------|-----------------------|--------------------|
| Train         | 1404                  | 36                 |
| Test          | 351                   | 36                 |
| Всего         | 1755                  | 36                 |

Разбиение выполнялось стратифицированно. Для данной задачи это особенно важно, так как в датасете присутствуют как крупные классы, содержащие сотни документов, так и редкие классы, представленные единичными или малочисленными примерами. Если при разделении данных какой-либо редкий класс исчез бы из тестовой выборки, итоговая оценка не отражала бы качество классификации по полному множеству классов. На рисунках 2.6, 2.7 проиллюстрировано финальное распределение классов.

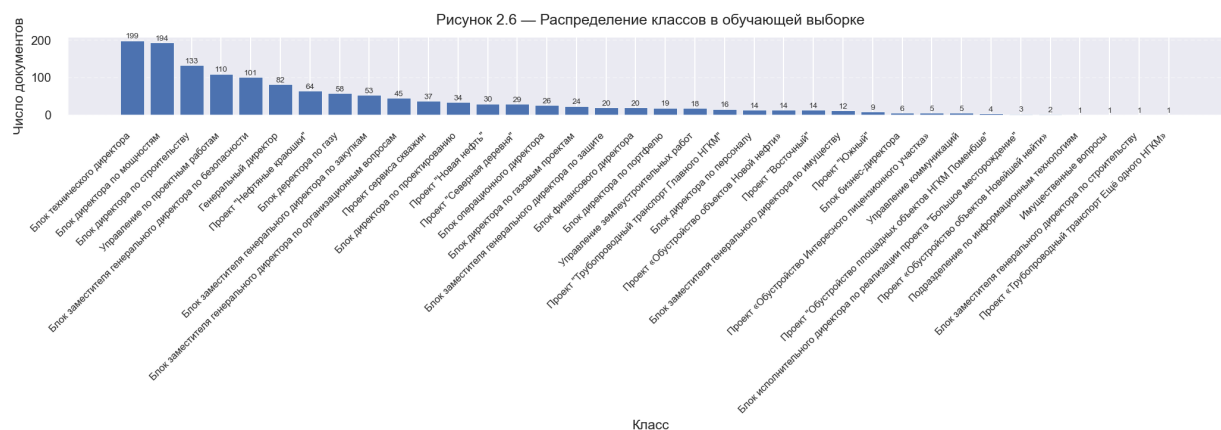


Рисунок 2.6 — Распределение классов в обучающей выборке

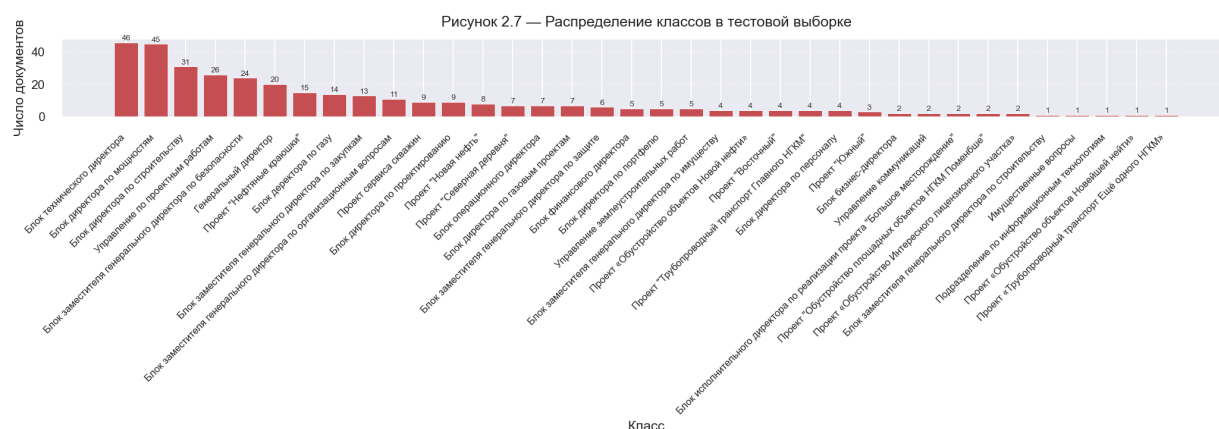


Рисунок 2.7 — Распределение классов в тестовой выборке

Несмотря на сохранение всех классов в обеих частях, распределение остаётся несбалансированным. Крупные категории представлены значительно большим числом документов, чем редкие. Такая структура соответствует реальному характеру деловой переписки: по одним направлениям поток писем формируется постоянно, тогда как другие проекты или подразделения встречаются заметно реже. Поэтому дисбаланс не рассматривался как ошибка данных, а учитывался как одно из условий задачи.

Особенность выбранного разбиения заключается также в том, что тестовая выборка не изменялась на последующих этапах работы. Аугментация, суммаризация документов и другие способы расширения данных применялись только к train-части. Это позволило избежать утечки информации из теста в обучение и сохранить корректность сравнения разных моделей. Все методы оценивались на одной и той же тестовой выборке, что делает результаты сопоставимыми между собой.

### **3 Постановка задачи и проектирование решения**

#### **3.1 Формальная постановка задачи многоклассовой классификации**

В рамках работы требуется построить программный пайплайн, который по тексту делового письма автоматически определяет его принадлежность к одному из заданных классов. На вход системе подаётся текст документа после этапа предобработки, на выходе формируется метка класса. Предсказание выполняется в режиме single-label классификации: для каждого письма выбирается только одна итоговая категория.

Задача рассматривается в постановке обучения с учителем. Для обучающих объектов известны правильные метки, поэтому модели обучаются на парах «текст — класс». После обучения классификатор должен обобщать выявленные закономерности на новые письма, которые не использовались при настройке модели. Все сравниваемые подходы решают одну и ту же задачу, но отличаются способом преобразования текста в признаки и типом классификационной модели.

С инженерной точки зрения задача реализуется как последовательность этапов: предобработка текста, построение признакового представления, обучение классификатора и получение предсказаний. В качестве признаков могут использоваться разреженные TF-IDF-векторы, плотные BERT-эмбединги, суммаризованные варианты текста и композитные векторы, объединяющие разные типы представлений. Это позволяет не ограничиваться одной моделью, а последовательно сравнивать несколько решений в единой постановке задачи.

#### **3.2 Требования к функциональности и воспроизводимости**

Разрабатываемое решение должно обеспечивать полный цикл обработки текстов: от подготовки исходных писем до обучения моделей и получения итоговых метрик. Поскольку задача относится к прикладной классификации деловой переписки, важна не только точность, но и воспроизводимость экспериментов. Для каждого этапа должны быть зафиксированы используемые данные, правила предобработки, параметры моделей и набор метрик, чтобы результаты можно было повторить и сопоставить при изменении отдельных компонент пайплайна.

### **3.3 Влияние предметной области на проектирование решения**

Предметная область определяет выбор методов обработки и классификации текстов. В рассматриваемой задаче анализируются длинные деловые письма, связанные с проектами, подразделениями и управленческими направлениями. Для таких документов важны как точные лексические совпадения (названия подразделений, аббревиатуры, формальные ссылки), так и общий контекст письма. Дополнительной сложностью является тематическая близость многих классов: письма из разных категорий могут использовать схожий деловой язык — обсуждение согласований, сроков, подготовки материалов и организационных вопросов. Это требует от модели умения различать тонкие смысловые различия между близкими по формулировкам обращениями.

### **3.4 Архитектура пайплайна обработки текста**

Решение рассматривается как последовательность взаимосвязанных этапов, через которые проходит каждое письмо: от исходного текста до отнесения к одному из классов. На концептуальном уровне это цепочка преобразований, где на каждом шаге текст получает более структурированное и информативное представление, а в финале используется модель, способная по этому представлению принять решение о классе.

Такая архитектура задаёт каркас, внутри которого можно по отдельности выбирать и менять конкретные методы предобработки, аугментации, построения признаков и классификации. Это позволяет не привязываться к одному фиксированному решению, а проектировать систему как набор взаимозаменяемых компонентов, оценивая вклад каждого из них в итоговое качество классификации деловых писем.

### **3.5 Выбор направлений моделирования**

Первым направлением стали классические модели на основе TF-IDF-признаков. Этот подход был выбран как базовая линия, поскольку он хорошо подходит для задач классификации текстов на небольших и средних выборках, быстро обучается и позволяет использовать как словные, так и символьные n-граммы. В работе отдельно рассматриваются word-признаки, char-признаки и их объединение. Поверх полученных векторов обучаются линейные классификаторы, в том числе LinearSVC и LogisticRegression.

Вторым направлением стали BERT-подобные модели. Они были выбраны для проверки того, насколько контекстные представления помогают в задаче классификации



длинных русскоязычных деловых писем. В рамках этого направления рассматриваются ruBERT и ruRoberta-large, а также разные варианты использования выхода модели: классификация с использованием CLS-токена, mean pooling и полноценный классификатор для длинных документов с чанкингом.

Третьим направлением стало построение эмбединговых представлений документов. В отличие от end-to-end классификации BERT, здесь модель используется для получения векторов текстов, после чего поверх этих векторов обучаются отдельные классификаторы. Такой подход позволяет отделить этап построения признаков от этапа классификации и проверить, насколько информативными являются плотные представления для данной предметной области.

Четвёртым направлением стали композитные признаки. В этом варианте разреженные TF-IDF-векторы объединяются с плотными BERT-представлениями. Идея заключается в том, чтобы совместить точные лексические признаки, полезные для деловых текстов, и контекстную информацию, извлекаемую нейросетевыми моделями. Для классификации таких объединённых векторов рассматриваются классические модели и MLP-классификатор.

### **3.6 Использование суммаризации для подготовки дополнительных представлений текста**

Использование суммаризации позволяет проверить гипотезу о том, что сокращённая версия письма может быть более компактным и менее шумным представлением документа. Если суммаризация сохраняет ключевую информацию, классификатор может получать более сфокусированный текст, в котором меньше повторов и технических элементов. Это особенно важно для BERT-подобных моделей, у которых есть ограничение на длину входной последовательности.

В работе суммаризация рассматривается как дополнительный этап подготовки данных, а не как замена основной предобработки.

### **3.7 Построение суммаризованного и комбинированного наборов данных**

Для проверки влияния суммаризации были сформированы дополнительные варианты набора данных. Первый вариант основан только на суммаризованных текстах. В этом случае вместо полного письма в модель передаётся его краткое содержание. Такой

набор позволяет проверить, достаточно ли summary для сохранения признаков, необходимых для определения класса.

Второй вариант является комбинированным. В нём исходный текст объединяется с его суммаризованной версией. Такой подход выбран для того, чтобы не терять детали оригинального письма, но одновременно добавить к нему более компактное представление основного содержания. Комбинированный вариант можно рассматривать как способ обогащения текста: модель получает и полный документ, и его краткое смысловое описание.

При построении таких наборов важно сохранять соответствие между текстом и исходной меткой класса. При этом тестовая часть не используется для обучения моделей суммаризации или настройки классификаторов. Все варианты данных затем оцениваются в едином экспериментальном протоколе.

### **3.8 Выбор метрик качества классификации**

Для оценки качества моделей в работе используются метрики, подходящие для многоклассовой классификации при несбалансированном распределении классов. В такой задаче недостаточно ориентироваться только на ассигасу, поскольку высокая общая точность может достигаться за счёт правильной классификации крупных классов, при этом качество на редких категориях может оставаться низким.

Основными метриками выбраны balanced accuracy и macro F1. Balanced accuracy позволяет учитывать качество распознавания по каждому классу и снижает влияние доминирующих категорий. Macro F1 также важна для данной задачи, так как рассчитывается с равным вкладом каждого класса и показывает, насколько устойчиво модель работает не только на крупных, но и на малых категориях.

Выбор таких метрик связан с особенностями исходной задачи. Поскольку в данных присутствуют классы с разным количеством документов, итоговая оценка должна показывать не только способность модели распознавать наиболее частые категории, но и её устойчивость на всём множестве классов. Поэтому в финальном сравнении основное внимание уделяется balanced accuracy и macro F1, а остальные показатели используются для дополнительной интерпретации результатов.

## **4 Аугментация данных и обогащение выборки**

### **4.1 Мотивация аугментации при дисбалансе классов**

Как показал разведочный анализ данных, присутствуют классы, содержащие всего один пример в обучающей выборке. Также присутствуют менее критичные случаи, но без аугментации у моделей не будет шансов определять класс для таких документов.

### **4.2 Аугментация классическими методами без использования нейросетевых моделей**

Случайное удаление слов, замена синонимами, случайная перестановка слов и другие методы не показали улучшения метрик на методах классификации и в итоговый пайплайн не вошли.

### **4.3 Общая схема нейросетевой аугментации длинных текстов**

Основная аугментация в работе выполняется двумя нейросетевыми методами: перефразированием и обратным переводом. Несмотря на различие самих моделей, общая схема обработки для этих методов имеет сходную структуру. Это позволяет применять единый подход к длинным деловым письмам и контролировать качество получаемых текстов.

Сначала в тексте маскируются чувствительные и структурно важные фрагменты, такие как персоналии, организации и даты. Для этого используются специальные placeholder-обозначения. Маскирование необходимо для того, чтобы генеративная модель не искажала такие элементы и не создавала некорректные варианты имён, организаций или временных выражений.

Затем документ разделяется на фрагменты. Полученные тексты с разными гиперпараметрами подаются нейросетевым моделям. Происходит генерация нескольких примеров с выбором лучшего из них. Полученные новые части документа конкатенируются обратно. Происходит обратный процесс восстановления масок.

Полученный аугментированный текст сравнивается на схожесть со всеми оригинальными документами класса. Если проверка пройдена, пример добавляется, иначе алгоритм переключается на следующий оригинальный документ этого же класса для аугментации. Если не удалось за фиксированное число повторений составить ни одного нового примера, то алгоритм переключается на следующий класс. Это нужно, чтобы не

зацикливаться на сложных примерах и вернуться к ним позже, после того как будут обработаны все простые примеры.

#### **4.4 Разбиение длинных текстов для аугментации**

При нейросетевой аугментации длинное письмо сначала разбивается на более короткие, осмысленные части. Основой служит разбиение на предложения, так как в деловой переписке одно предложение часто соответствует отдельному поручению, просьбе или описанию ситуации. Если предложение получается слишком длинным для генеративной модели, оно дополнительно делится на более компактные фрагменты по знакам пунктуации, а при необходимости — по словам, чтобы получить отрезки удобной длины для обработки.

Если предложение невозможно корректно разделить по пунктуации, применяется аварийное разбиение на окна фиксированного размера. Такой механизм нужен для устойчивости пайплайна: даже нестандартные или плохо структурированные фрагменты письма должны быть обработаны без остановки всей аугментации.

В результате каждый исходный документ представляется как последовательность фрагментов. Каждый фрагмент аугментируется отдельно, после чего итоговые варианты объединяются обратно. Это позволяет применять перефразирование и обратный перевод к длинным письмам, не теряя структуру документа полностью.

#### **4.5 Генерация нескольких вариантов аугментации и выбор итогового кандидата**

Для каждого фрагмента текста генерируется не один, а несколько вариантов аугментации. Это необходимо потому, что результат генеративной модели может отличаться по качеству: один вариант может быть слишком близким к исходному тексту, другой — слишком свободным, третий — содержать неудачную формулировку. Генерация нескольких кандидатов позволяет выбрать наиболее подходящий вариант для включения в итоговый документ.

В методе обратного перевода используются несколько языков-посредников и разные режимы генерации. Это даёт набор альтернативных формулировок для одного и того же исходного фрагмента. В методе перефразирования также используются несколько режимов генерации, включая beam search и sampling. За счёт этого модель может создавать варианты с разной степенью близости к исходной формулировке.

Выбор кандидата выполняется по набору критериев. На первом уровне проверяется техническая пригодность текста: кандидат не должен быть пустым, чрезмерно коротким или полностью совпадать с оригиналом. Далее оценивается близость к исходному фрагменту. Для этого используется косинусное сходство между эмбедингами исходного и сгенерированного текста. Подходящий кандидат должен сохранять смысл исходного фрагмента, но при этом не быть его точной копией.

Для метода обратного перевода дополнительно учитывается естественность текста, оцениваемая с помощью перплексии языковой модели. Это позволяет отсеивать менее удачные варианты и выбирать более гладкие и грамматически корректные формулировки. В итоге в итоговый документ включаются не все сгенерированные фрагменты, а только те, которые прошли фильтрацию и удовлетворяют заданным требованиям к качеству.

#### **4.6 Схема доведения редких классов до порогового размера выборки**

Аугментация в работе в первую очередь направлена на увеличение числа примеров в редких классах. Вместо равномерного расширения всей выборки используется более точечный подход: для каждого класса оценивается текущее количество документов, и новые примеры генерируются только там, где данных недостаточно. Это позволяет не перегружать датасет лишними синтетическими текстами и сосредоточиться на действительно проблемных категориях.

В качестве целевого порога выбрано значение 30 документов на класс. Если класс содержит меньше примеров, запускается генерация дополнительных текстов на основе имеющихся документов. При этом исходные тексты используются циклически, а число попыток генерации ограничено, чтобы избежать бесконечного процесса в случаях, когда не удаётся получить качественные варианты.

Каждый сгенерированный текст проходит несколько этапов проверки. Контролируется его длина, отличие от исходного документа и степень смысловой близости, измеряемая через косинусное сходство эмбедингов. Дополнительно оценивается, насколько новый текст соответствует пространству своего класса — он должен быть близок не только к исходному документу, но и к другим примерам той же категории. Это снижает риск попадания в обучающую выборку искажённых или нехарактерных текстов.

Такая стратегия позволяет целенаправленно выравнивать распределение классов, не увеличивая объём данных без необходимости. Для несбалансированного датасета это

особенно важно: простое равномерное увеличение всех классов привело бы к росту корпуса, но не дало бы существенного улучшения качества.

Отдельным преимуществом является автоматизация всего процесса. Пайплайн аугментации можно запускать повторно на любом датасете с похожей структурой. В рамках работы он использовался в двух сценариях: студент запускал его на обезличенных данных, а научный руководитель — на исходном наборе с конфиденциальной информацией, при этом логика обработки оставалась полностью одинаковой.

#### **4.7 Аугментация с использованием перефразирования**

Перефразирование используется для получения новых вариантов текста с сохранением исходного смысла. В этом подходе модель получает фрагмент письма и генерирует альтернативную формулировку на русском языке. Такой метод хорошо подходит для задачи классификации, так как позволяет увеличить разнообразие выражений внутри одного класса без изменения целевой метки.

В работе для перефразирования использовалась модель семейства `ruT5`. Она применялась как генеративная `seq2seq`-модель: на вход подавался исходный фрагмент текста, а на выходе формировался его перефразированный вариант. Использование русскоязычной модели важно для данной задачи, поскольку исходные документы представляют собой деловые письма на русском языке, содержащие специфичные формулировки, названия проектов и служебные конструкции.

#### **4.8 Аугментация методом обратного перевода**

Обратный перевод является вторым нейросетевым методом аугментации, использованным в работе. Его идея заключается в том, что исходный русский текст переводится на промежуточный язык, а затем обратно на русский. При таком преобразовании часть формулировок изменяется, но общий смысл должен сохраняться. Это позволяет получить дополнительный вариант текста без ручного составления правил перефразирования.

Для реализации обратного перевода использовались модели машинного перевода семейства `Helsinki-NLP/OPUS-MT`. Перевод выполнялся по нескольким направлениям: сначала с русского языка на промежуточный язык, затем обратно на русский. В качестве промежуточных языков использовались английский, французский и испанский, что

позволяло получать несколько различных вариантов переформулирования одного и того же фрагмента.

Метод обратного перевода, как правило, даёт более естественные перефразирования по сравнению с простыми случайными заменами слов, однако может вносить искажения. В частности, отдельные термины переводятся неточно, деловой стиль может становиться менее строгим, а названия и аббревиатуры — изменяться или терять исходный вид. Чтобы снизить эти эффекты, перед переводом ключевые сущности заменяются специальными placeholder-маркерами, что позволяет сохранить корректность предметной лексики и избежать нежелательных преобразований.

#### **4.9 Отбор аугментированных текстов по косинусному сходству**

Фильтрация по косинусному сходству применяется как основной механизм контроля качества аугментированных текстов. После генерации важно убедиться, что новый вариант сохраняет смысл исходного документа, но не превращается в его почти дословную копию. Для этого исходный и аугментированный текст переводятся в векторное пространство, после чего между ними считается косинусное сходство. В работе для получения эмбедингов используется модель `deepvk/USER2-base`; эмбединги нормализуются, а сходство оценивается не только по совпадению слов, но и по близости смыслового содержания. Для отбора задаётся допустимый диапазон: слишком низкое сходство говорит о потере смысла, слишком высокое — о том, что текст почти не отличается от оригинала и не добавляет разнообразия. В аугментацию включаются только те варианты, которые попадают в этот диапазон. Дополнительно проверяется близость кандидата к другим текстам того же класса, чтобы контролировать не только связь с исходным документом, но и соответствие общей теме класса, что особенно важно для редких категорий.

#### **4.10 Восстановление полного текста из аугментированных фрагментов**

Поскольку аугментация выполняется на уровне отдельных фрагментов, после генерации их нужно собрать обратно в полный документ. Для этого выбранные аугментированные фрагменты объединяются в исходном порядке, чтобы сохранить структуру письма и последовательность смысловых частей. Затем выполняется демаскирование: специальные обозначения, которыми временно заменялись имена, названия и другие важные сущности, возвращаются к исходным значениям. После

объединения выполняется лёгкая постобработка: удаляются лишние пробелы, устраняются технические артефакты генерации и текст приводится к единому формату корпуса. Этот этап важен, поскольку модели классификации работают с целостными документами, и итоговый аугментированный пример должен представлять собой связный текст, а не набор отдельных фрагментов.

#### 4.11 Итоговый аугментированный обучающий набор

После генерации перефразированных и обратно переведённых текстов были сформированы отдельные аугментированные обучающие наборы для каждого метода, что позволило независимо оценить их вклад. Затем полученные данные объединили в единый набор, выполнили дедупликацию по паре «метка класса — нормализованный текст» для устранения повторов и перемешали с фиксированным значением генератора случайных чисел для обеспечения воспроизводимости.

Таблица 4.1 — Наборы данных после аугментации

| Набор данных            | Количество объектов | Описание                                 |
|-------------------------|---------------------|------------------------------------------|
| train.csv               | 1404                | Исходная обучающая выборка после очистки |
| train_backtranslate.csv | 1816                | Обучающая выборка с обратным переводом   |
| train_paraphrase.csv    | 1825                | Обучающая выборка с перефразированием    |
| train_augmented.csv     | 2158                | Объединённый аугментированный набор      |

Помимо объединённого набора, были подготовлены дополнительные варианты данных для экспериментов с суммаризацией. Во всех случаях тестовая выборка оставалась неизменной, что позволяет оценивать различные модели и стратегии обучения на одном и том же наборе документов и корректно сравнивать их качество.

#### 4.12 Анализ распределения данных после аугментации

После формирования аугментированного набора был выполнен анализ распределения классов. Цель этого этапа заключалась в том, чтобы проверить, насколько аугментация изменила структуру обучающей выборки и удалось ли увеличить количество



примеров в редких категориях, распределение классов после аугментации представлено на рисунке 4.1. Поскольку генерация выполнялась преимущественно для малых классов, распределение после аугментации должно стать более сбалансированным по сравнению с исходным train-набором.

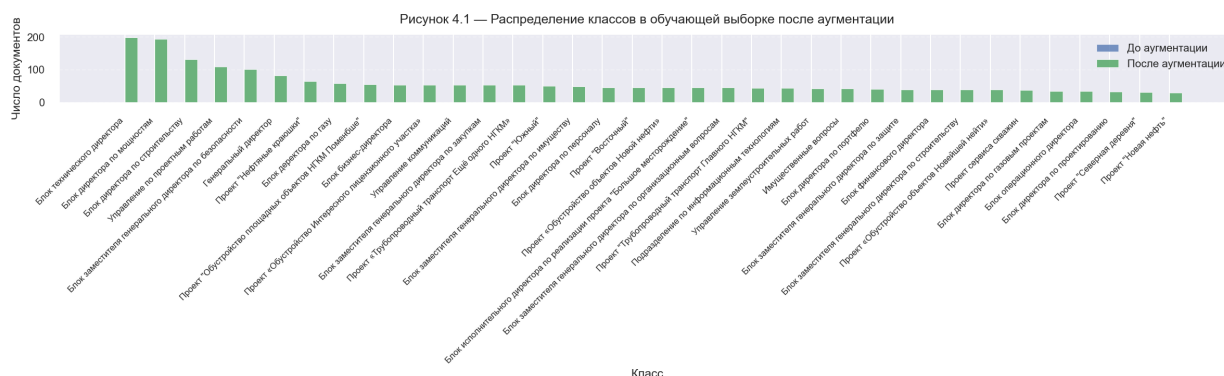


Рисунок 4.1 — Распределение классов в обучающей выборке после аугментации

Анализ показал, что большинство редких классов было увеличено до заданного порогового уровня или приблизилось к нему. При этом крупные классы не размножались пропорционально, что позволило избежать чрезмерного роста уже доминирующих категорий. Таким образом, аугментация была направлена именно на выравнивание обучающего распределения, а не на простое увеличение общего числа документов.

По сравнению с исходной обучающей выборкой объём данных увеличился, при этом все документы сохраняют одну из исходных 36 меток классов. Это позволяет использовать аугментированный набор в тех же моделях классификации, что и исходный train-набор.

Итоговый аугментированный набор далее используется в экспериментах с классическими моделями, BERT-подходами, эмбединговыми и композитными представлениями. Это позволяет оценить влияние обогащения данных на разные типы классификаторов в едином сравнительном протоколе.

## **5 Классические модели на основе разреженных текстовых признаков**

### **5.1 Теоретические основы TF-IDF-представления текста**

TF-IDF является одним из базовых способов преобразования текста в числовое признаковое пространство. В таком представлении документ описывается через набор токенов или n-грамм, а каждому признаку сопоставляется вес. Чем чаще признак встречается в конкретном документе и чем реже он встречается во всём корпусе, тем более информативным он считается для данного документа.

Для задачи классификации деловых писем TF-IDF-подход имеет несколько преимуществ. Во-первых, он хорошо работает с небольшими и средними по размеру наборами данных, где сложные нейросетевые модели могут не получить достаточного количества примеров для устойчивого дообучения. Во-вторых, TF-IDF позволяет явно учитывать слова, словосочетания, служебные обозначения и характерные фрагменты, которые могут быть связаны с конкретным классом. В деловой переписке такими признаками могут быть названия проектов, подразделений, типовые формулировки, внутренние аббревиатуры и устойчивые конструкции.

В отличие от плотных эмбедингов, TF-IDF формирует разреженное пространство признаков. Это означает, что каждому документу соответствует вектор большой размерности, где большинство значений равно нулю. Такая структура естественна для текстов, поскольку один документ содержит только небольшую часть всех возможных слов и n-грамм корпуса. При этом линейные классификаторы хорошо приспособлены к работе с такими признаками и позволяют быстро обучаться даже на больших разреженных матрицах.

В данной работе TF-IDF используется как основа классической ветки моделирования. На его базе строятся признаки разных типов: словные n-граммы, символьные n-граммы и их объединение. Такой набор вариантов позволяет проверить, какие уровни текстовой информации оказываются наиболее полезными для классификации длинных русскоязычных деловых писем.

### **5.2 Линейные классификаторы для разреженных текстовых признаков**

Для классификации TF-IDF-признаков в работе используются линейные модели. Их выбор связан с тем, что они хорошо работают с высокоразмерными разреженными представлениями и обычно дают устойчивый результат в задачах классификации текстов.

Линейная модель обучает вес для каждого признака и на основе взвешенной суммы признаков принимает решение о принадлежности документа к классу.

Основными классификаторами в классической ветке являются LinearSVC и LogisticRegression. LinearSVC основан на линейной версии метода опорных векторов и хорошо подходит для задач, где данные представлены большим количеством разреженных признаков. LogisticRegression также является линейной моделью, но интерпретирует задачу через оценку вероятностей классов. Оба подхода позволяют эффективно работать с TF-IDF-представлениями и быстро обучаются на CPU.

С учётом дисбаланса классов в обеих моделях учитывается различие в количестве объектов по классам, чтобы уменьшить смещение модели в сторону наиболее крупных категорий. Это особенно важно для рассматриваемой задачи, где часть классов представлена существенно меньшим числом документов.

### **5.3 Признаки на основе словных n-грамм**

Первый вариант TF-IDF-представления формируется на основе словных n-грамм. В этом подходе признаками выступают как отдельные слова, так и их последовательности, что позволяет учитывать не только наличие конкретных терминов, но и характерные сочетания слов, типичные для разных классов.

В работе используются униграммы и биграммы. Униграммы отражают отдельные слова и позволяют фиксировать базовую лексику документов, тогда как биграммы помогают учитывать устойчивые словосочетания, включая названия направлений, типовые деловые формулировки и фразы, специфичные для определённых категорий.

### **5.4 Признаки на основе символьных n-грамм**

Второй вариант TF-IDF-представления формируется на основе символьных n-грамм, где в качестве признаков используются не слова, а подстроки фиксированной длины. Такой подход особенно эффективен для русского языка, поскольку позволяет частично учитывать морфологию, включая окончания, приставки и вариации написания. Символьные n-граммы формируются внутри границ слов, что снижает количество случайных признаков, возникающих на стыках слов, и делает представление более устойчивым. Для ограничения словаря отбрасываются слишком редкие фрагменты, встречающиеся лишь в единичных документах, а также чрезмерно частые n-граммы, присутствующие почти во всём корпусе и слабо различающие классы. В контексте деловой переписки символьные признаки

особенно полезны из-за большого количества аббревиатур, служебных обозначений и проектных названий: даже при различиях полных слов совпадения на уровне символов дают модели дополнительную информацию для классификации.

## **5.5 Объединение словных и символьных признаков**

Словные и символьные признаки дополняют друг друга, поэтому в работе используется их совместное представление в одном пространстве. Словные  $n$ -граммы фиксируют отдельные термины и характерные словосочетания, а символьные  $n$ -граммы улавливают внутреннюю структуру слов и остаются устойчивыми к изменениям формы, сокращениям и вариантам написания. Для русскоязычных деловых писем такое сочетание особенно полезно, так как в текстах одновременно встречаются предметные термины, аббревиатуры, служебные обозначения и разнообразные грамматические формы.

Для объединения признаков включаются два независимых векторизатора: word TF-IDF и char TF-IDF. Каждый из них строит собственное признаковое пространство, после чего полученные матрицы объединяются в общий разреженный вектор документа.

В итоговом пайплайне объединённое представление используется как единое пространство признаков, передаваемое в классификатор. Это позволяет одновременно учитывать как точные словные совпадения, так и более устойчивые символьные паттерны. Такой подход особенно полезен в рассматриваемой предметной области, где одни классы определяются конкретными терминами и формулировками, а другие — более общими лексическими особенностями.

## **5.6 Реализация пайплайна TF-IDF + LinearSVC**

Основным классическим baseline в работе выбран пайплайн TF-IDF + LinearSVC, представляющий собой последовательность этапов построения признаков и обучения линейного классификатора. Такой подход используется как сильная и вычислительно эффективная базовая линия для задачи многоклассовой классификации текстов.

В качестве признакового пространства применяется объединение словных и символьных признаков: word-компонента строится на униграммах и биграмах, а char-компонента — на символьных  $n$ -граммах длины от 3 до 5. Полученные разреженные представления объединяются и подаются на вход классификатору LinearSVC.

Использование сбалансированных весов классов связано с неравномерным распределением документов в обучающей выборке. В этом режиме модель учитывает

частоты классов и увеличивает вклад редких категорий при обучении. Это позволяет сделать baseline более устойчивым к дисбалансу.

Данный пайплайн имеет несколько практических преимуществ. Он быстро обучается на CPU, не требует GPU и хорошо масштабируется на разреженные признаки. Кроме того, его удобно использовать как контрольную точку для сравнения с более сложными подходами: BERT-моделями, эмбединговыми классификаторами и композитными представлениями.

### **5.7 Реализация пайплайна TF-IDF + LogisticRegression**

Второй линейный классификатор, использованный в классической ветке, — LogisticRegression. Он применяется поверх того же TF-IDF-представления, что и LinearSVC, что позволяет сравнить влияние классификационной модели при одинаковом способе построения признаков.

LogisticRegression используется как альтернативная линейная модель к LinearSVC поверх тех же TF-IDF-признаков, чтобы проверить, насколько результаты зависят от выбора классификатора, а не только от признакового пространства. В отличие от LinearSVC, который строит разделяющие гиперплоскости с максимальным зазором, LogisticRegression обучается в вероятностной постановке, что даёт другой способ оптимизации и другую интерпретацию выходов. В работе она рассматривается не как замена основному baseline, а как дополнительный классификатор, позволяющий оценить устойчивость и качество TF-IDF-представления независимо от конкретного алгоритма.

### **5.8 Особенности TF-IDF для длинных деловых писем**

Применение TF-IDF к длинным деловым письмам имеет свои сильные стороны и ограничения. Важное преимущество в том, что такой подход не накладывает жёстких ограничений на длину текста: в отличие от BERT-подобных моделей, ограниченных числом токенов, TF-IDF может учитывать весь документ целиком. Для деловой переписки это критично, так как важные детали могут встречаться не только в начале письма, но и в середине или ближе к концу.

Для деловых писем также полезно то, что TF-IDF хорошо фиксирует характерные лексические признаки. Внутренние названия, проектные обозначения, аббревиатуры, типовые фразы и устойчивые словосочетания могут быть напрямую отражены в

признаковом пространстве. При использовании word и char n-граммные представлений модель получает доступ как к полным словам, так и к их фрагментам.

Однако длинные документы также создают сложности. Они могут содержать большое количество служебных и повторяющихся фрагментов: подписи, пересланную переписку, реквизиты, списки приложений, технические строки. Если такие элементы не обработаны на этапе предобработки, они могут увеличивать размер словаря и создавать шумовые признаки. Поэтому качество TF-IDF-подхода во многом зависит от предварительной очистки данных.

В данной работе TF-IDF используется не только как простой baseline, но и как важная часть общего исследования текстовых представлений. Сначала он применяется в классических моделях, затем его признаки используются при построении композитного векторного пространства вместе с BERT-представлениями. Это позволяет рассматривать TF-IDF как самостоятельный метод и как компонент более сложного признакового описания документа.

## **6 Контекстные представления и классификация на основе BERT-моделей**

### **6.1 Теоретические основы transformer-архитектур**

Transformer-архитектуры занимают центральное место в современной обработке естественного языка благодаря механизму self-attention, который позволяет каждому токenu входной последовательности одновременно учитывать все остальные токены. В отличие от рекуррентных сетей, обрабатывающих текст пошагово, transformer строит контекстуализированное представление каждого токена за один проход через стек энкодер-блоков, каждый из которых содержит многоголовочный self-attention и послойную нормализацию. Это даёт ключевое преимущество: представление каждого слова формируется с учётом его контекста в конкретном предложении, а не на основе усреднённой статистики по корпусу.

BERT-подобные модели представляют собой encoder-архитектуры transformer, предварительно обученные на больших текстовых массивах с использованием задач восстановления замаскированных токенов и определения связности предложений. В результате обучения модель усваивает широкий спектр языковых закономерностей, включая синтаксис, морфологию и семантику. При дообучении на прикладной задаче эти знания адаптируются к конкретной предметной области, что делает такие модели эффективными даже при ограниченном объёме размеченных данных.

В данной работе transformer-модели применяются для классификации длинных русскоязычных деловых писем. При этом принадлежность документа к классу часто определяется не отдельными словами, а их сочетанием в контексте: одно и то же слово в разных письмах может указывать на различные направления деятельности. Кроме того, стандартные BERT-подобные модели ограничены фиксированной максимальной длиной входной последовательности, что требует специальных решений для документов, превышающих этот предел. Именно поэтому в работе реализованы как базовые варианты классификации с усечением текста, так и отдельный классификатор с разбиением документа на перекрывающиеся фрагменты.

### **6.2 Контекстные эмбединги и их отличие от разреженных признаков**

TF-IDF даёт разреженное представление текста: каждый признак соответствует конкретному слову или n-грамме, и модель опирается на явные лексические совпадения и устойчивые формулировки. При этом одно и то же слово всегда кодируется одинаково,

независимо от контекста, а близкие по смыслу фразы могут выглядеть как совершенно разные наборы признаков. Контекстные BERT-эмбединги работают иначе: текст пропускается через нейросеть целиком, и вектор документа зависит от окружения слов. Одно и то же слово в разных контекстах получает разные представления, что позволяет различать значения и улавливать смысловые нюансы, важные для деловой переписки, где классы часто отличаются не отдельными словами, а типичными формулировками и описанием ситуации.

Оба типа признаков взаимодополняют друг друга. Разреженные признаки обеспечивают высокую интерпретируемость и стабильность на точных формулировках; контекстные — способность к обобщению по семантически близким текстам. В данной работе BERT-представления исследуются как самостоятельный подход в нескольких режимах: прямая end-to-end классификация, построение документных эмбедингов для обучения классических моделей и как компонент композитного признакового пространства.

### **6.3 Используемые русскоязычные модели**

Для работы с русскоязычными текстами были выбраны модели, предобученные на корпусах, включающих русский язык. В экспериментах использовались две основные модели и одна вспомогательная.

DeepPavlov/rubert-base-cased — русскоязычная BERT-модель базового размера. Её регистрозависимая токенизация важна для деловой переписки с аббревиатурами и наименованиями подразделений. Модель применяется в базовых экспериментах с разными классификационными головами при умеренных вычислительных требованиях.

ai-forever/ruRoberta-large — крупная RoBERTa-модель, оптимизированная для русского языка. По сравнению с базовым BERT она имеет большую размерность скрытых состояний, более мощный encoder и обучалась по улучшённому протоколу без задачи предсказания следующего предложения. Эта модель используется во всех ветках работы: базовая классификация без чанкинга, Sentence-BERT-дообучение для получения эмбедингов, построение композитных признаков и основной классификатор длинных документов.



Таблица 6.1 — Используемые BERT- и SBERT-модели

| Модель                       | Назначение                                              |
|------------------------------|---------------------------------------------------------|
| DeepPavlov/rubert-base-cased | Эксперименты с ruBERT-классификацией                    |
| ai-forever/ruRoberta-large   | Классификация, эмбединги, чанкинг, композитные признаки |

## 6.4 Два подхода к использованию BERT в задаче классификации

В работе реализованы два принципиально разных способа применения BERT-подобных моделей.

Первый подход — использование BERT как генератора документных эмбедингов. Здесь модель преобразует документ в плотный вектор, который затем передаётся в отдельный классификационный алгоритм: линейный классификатор, метод ближайшего соседа или модель на основе косинусного сходства. Такой подход допускает гибкое использование одних и тех же эмбедингов в разных экспериментальных ветках без повторного дообучения encoder. Эмбединги строятся один раз и затем применяются в нескольких экспериментах: обучение классических классификаторов, косинусная классификация, композитные признаковые пространства.

Второй подход — дообучение BERT как end-to-end классификатора. В этом варианте encoder и классификационная голова обучаются совместно по функции потерь классификации. Все параметры модели, включая параметры transformer-блоков, обновляются под конкретную задачу, что потенциально даёт лучшую адаптацию к предметной области. Недостаток — более высокая вычислительная стоимость и риск переобучения на малой выборке. В этом режиме модель оптимизируется непосредственно под целевые классы, тогда как в первом подходе encoder остаётся фиксированным или дообучается отдельно на метрической задаче без прямого доступа к меткам классов.

Оба подхода реализованы применительно к задаче классификации длинных деловых писем, что позволяет сопоставить не только разные модели, но и разные стратегии использования одного и того же transformer-encoder.

### 6.4.1 Дообучение Sentence-BERT для построения эмбедингов

Для построения документных эмбедингов используется подход на основе библиотеки Sentence Transformers. Архитектура представляет собой стек из трёх компонентов: transformer-encoder, слой pooling и нормализация итогового вектора. В

качестве encoder используется ruRoberta-large. Pooling выполняется усреднением скрытых состояний всех токенов с учётом маски внимания, что позволяет каждой позиции последовательности вносить пропорциональный вклад в итоговый вектор. Нормализация выходного вектора выполняется так, чтобы косинусное сходство между документами можно было напрямую использовать как меру близости их эмбедингов, что упрощает последующую классификацию и ранжирование.

Обучение выполняется на парах фрагментов текста, сформированных из обучающей выборки. Документы одного класса образуют позитивные пары, документы разных классов — негативные. Важно, что в качестве позитивов берутся только пары чанков из разных документов одного класса: фрагменты одного и того же письма не считаются позитивной парой, даже если относятся к одному классу. Это сделано для того, чтобы модель училась находить сходство между независимыми письмами одного типа, а не просто запоминала соседние куски одного и того же текста. Такой подход помогает избежать коллапса представлений и заставляет encoder выделять признаки, общие для всех документов класса.

В качестве функции потерь используется MultipleNegativesRankingLoss (MNR), которая формулирует обучение как задачу ранжирования. Для каждой позитивной пары все остальные примеры в батче автоматически выступают в роли негативов. Модель оптимизируется так, чтобы вектор якорного примера был ближе к вектору своего позитивного партнёра, чем ко всем остальным в батче. Это позволяет эффективно использовать каждый батч без явного построения большого числа негативных пар в датасете и хорошо масштабируется при увеличении размера батча.

Перед построением пар документы разбиваются на чанки фиксированного размера с перекрытием. Длина чанка подбирается под максимальную длину входа модели, а перекрытие позволяет не терять контекст на границах фрагментов. Дополнительно для каждого письма формируется глобальный чанк, который объединяет начало и конец документа, чтобы учесть важные элементы, расположенные в разных частях текста (например, тему в начале и конкретную просьбу в конце). Позитивные пары строятся между чанками разных документов одного класса, за счёт чего объём обучающих данных увеличивается даже при ограниченном числе исходных писем.

На этапе инференса длинный документ обрабатывается тем же образом: он разбивается на чанки, каждый из них кодируется encoder отдельно, после чего векторы чанков агрегируются в один документный эмбединг. Рассматриваются несколько вариантов агрегации: усреднение векторов чанков, взятие поэлементного максимума или

их комбинация. Итоговый вектор нормализуется до единичной длины. Такой режим работы согласован с обучением: модель обучалась на чанках заданного формата и на инференсе получает входы того же типа.

#### **6.4.2 Дообучение BERT как сквозного классификатора**

В сквозной постановке (end-to-end) transformer-encoder дополняется классификационной головой, и вся модель обучается целиком на задачу предсказания класса. В этом случае обновляются не только веса головы, но и сами контекстные представления encoder, что позволяет подстроить внутренние эмбединги под конкретные классы и особенности деловых писем. Градиент от функции потерь напрямую влияет на encoder, поэтому модель может перестраивать свои представления так, чтобы лучше разделять категории в терминах целевой классификационной задачи.

В работе рассматриваются стандартные варианты без чанкинга и специализированная архитектура для длинных документов. Стандартные варианты проверяют базовые конфигурации с разными способами агрегации выхода encoder (токен CLS и mean pooling) при усечении текста до максимальной длины входа. Основным end-to-end подходом является chunked-классификатор на ruRoberta-large, учитывающий несколько частей длинного письма через разбиение на перекрывающиеся фрагменты с двухуровневой агрегацией. Детали реализации описаны в разделах 6.9–6.14.

#### **6.5 Классификация через специальный токен CLS**

Первый вариант end-to-end классификации основан на стандартной архитектуре последовательной классификации из библиотеки hugging face transformers. Transformer-encoder дополняется классификационной головой, принимающей скрытое состояние специального токена CLS и возвращающей логиты по всем классам. Благодаря механизму self-attention этот токен агрегирует информацию по всей входной последовательности и используется как её обобщённое представление.

Данный подход применяется к моделям ruBERT-base-cased и ruRoberta-large. Тексты проходят токенизацию и усекаются до максимально допустимой длины входа. Если документ превышает это ограничение, его часть отбрасывается. При этом стандартная архитектура требует минимальных изменений относительно базовой реализации HuggingFace и служит отправной точкой для сравнения с более сложными вариантами. Голова представляет собой линейный слой, проецирующий скрытое состояние CLS-токена в пространство логитов классов.

## **6.6 Классификация через усреднение скрытых состояний токенов (mean pooling)**

Альтернативный вариант агрегации выхода encoder — усреднение скрытых состояний всех токенов с учётом маски внимания. В отличие от использования специального токена CLS, mean pooling задействует информацию из каждой позиции последовательности пропорционально её наличию. Это может быть полезно, когда смысловая нагрузка распределена по всему тексту, а не сосредоточена в начале документа или в одном токене.

Реализация включает: передачу текста через encoder, поэлементное умножение скрытых состояний на маску внимания для исключения паддинговых позиций, суммирование по оси последовательности и деление на количество реальных (не паддинговых) токенов. Полученный усреднённый вектор проходит через слой регуляризации (dropout) и линейный классификатор. Архитектура применяется как для ruBERT-base-cased, так и для ruRoberta-large, что позволяет разделить влияние способа агрегации и выбранного encoder.

Ограничение на длину входа сохраняется так же, как и в стандартной архитектуре: текст усекается, если превышает допустимый предел. Поэтому mean pooling рассматривается как альтернативный способ использования выхода encoder, а не как решение проблемы длинных документов. Преимущество mean pooling — более равномерное использование информации из всех токенов последовательности, что теоретически должно улучшить обобщение, когда ключевые признаки класса распределены по документу.

## **6.7 Эксперименты с ruBERT-base-cased и различными классификационными головами**

Модель ruBERT-base-cased применяется как базовая русскоязычная BERT-модель с умеренными вычислительными требованиями. На её основе исследуются два варианта классификации — через стандартную голову с CLS-токеном и через mean pooling токенов. Оба варианта обучаются в одинаковых условиях: одно и то же разбиение данных, одни и те же метрики оценки, одинаковый оптимизатор с cosine-режимом планировщика скорости обучения и фазой warmup в начале обучения.

Эти эксперименты выполняют двойную функцию. Во-первых, они служат нейросетевой базовой линией — наименее сложным BERT-подходом, с которым сопоставляются более крупные и сложные модели. Во-вторых, сравнение двух вариантов головы внутри одной модели позволяет изолированно оценить вклад способа агрегации выхода encoder при прочих равных условиях. Если mean pooling даёт заметное улучшение относительно CLS-подхода, это указывает на распределённый характер информативных признаков в текстах; если разница минимальна — на достаточность представления CLS-токена.

Поскольку оба варианта работают с усечением длинных документов, результаты отражают качество классификации при неполной передаче текста и могут недооценивать потенциал BERT-подхода в задаче с длинными письмами. Эти эксперименты закладывают нижнюю границу качества нейросетевых подходов.

## **6.8 Эксперименты с ruRoberta-large и различными классификационными головами**

Модель ruRoberta-large тестируется в тех же архитектурных вариантах (стандартная голова с CLS и mean pooling), что и ruBERT. По сравнению с ruBERT-base-cased эта модель имеет большую размерность скрытых состояний, больше слоёв encoder и более качественно оптимизированные веса предобучения благодаря улучшённому обучающему протоколу без задачи предсказания следующего предложения. Это делает ruRoberta-large более мощным encoder по сравнению с ruBERT-base-cased, но одновременно увеличивает требования к вычислительным ресурсам: при ограничениях по памяти приходится уменьшать размер батча или применять дополнительные методы оптимизации памяти.

Эксперименты без чанкинга проводятся с усечением текста до максимально допустимой длины входа, поэтому для длинных документов часть информации неизбежно теряется. Такие эксперименты рассматриваются как базовая линия для ruRoberta-large перед использованием chunked-архитектуры и позволяют оценить, насколько более мощный encoder способен компенсировать потери, вызванные усечением текста. Если модель демонстрирует заметное улучшение по сравнению с ruBERT, это указывает на преимущество более глубоких представлений; в противном случае это подчёркивает значимость ограничения длины входа и необходимость разбиения документов на фрагменты.

## 6.9 Ограничения длины входной последовательности в BERT-моделях

BERT-подобные модели способны обрабатывать только ограниченное число токенов в одном проходе. Длина определяется архитектурой позиционных эмбеддингов и для используемых в работе моделей ruBERT и ruRoberta-large составляет 512 токенов. Для коротких текстов это ограничение не критично, однако деловые письма могут существенно превышать этот предел. Согласно анализу данных, значительная доля документов в исследуемом корпусе после очистки всё равно содержит больше токенов, чем допускает стандартный вход.

Простое усечение текста до максимальной длины ведёт к потере информации из средней и конечной части письма. Для деловой корреспонденции это особенно нежелательно: ключевой смысловой фрагмент может располагаться в любом месте текста. Если усечение отбрасывает именно ту часть письма, которая определяет класс, модель не сможет корректно классифицировать документ независимо от мощности encoder.

Именно поэтому базовые BERT-эксперименты без чанкинга используются лишь как отправная точка, а основное решение строится вокруг явного разбиения документа на перекрывающиеся фрагменты. Такой подход позволяет модели обработать весь документ, не нарушая ограничений архитектуры transformer.

### 6.10 Chunked-классификатор: архитектура для длинных документов

Основной классификатор длинных документов строится поверх ruRoberta-large и включает три уровня: encoder, механизм обработки чанков с двухуровневой агрегацией и классификационную голову.

Разбиение на перекрывающиеся чанки. Для обработки документов, превышающих ограничение длины входа, текст разбивается на несколько перекрывающихся фрагментов. Каждый чанк формируется из последовательности токенов заданной длины, равной максимально допустимой длине входа модели. Соседние чанки перекрываются на фиксированное число токенов, что реализуется через параметр stride при токенизации. Перекрытие гарантирует, что информация на границах фрагментов не теряется: фраза, разделённая двумя чанками при непересекающемся разбиении, полностью попадает хотя бы в один из них при скользящем окне.

Для ограничения вычислительной сложности число чанков на один документ ограничено сверху. Документы, генерирующие больше фрагментов, усекаются до максимального количества чанков. Если реальное число фрагментов меньше максимума,

позиции дополняются паддингом — чанками из паддинговых токенов. Маска числа чанков передаётся в модель как дополнительный вход, позволяя модели отличать содержательные фрагменты от пустых. Такая схема сохраняет единообразный размер тензоров в батче при переменной длине документов и позволяет эффективно обрабатывать батчи документов разной длины.

Двухуровневая агрегация представлений. После обработки каждого чанка encoder-моделью выполняется двухуровневая агрегация: сначала на уровне токенов внутри чанка, затем на уровне чанков внутри документа.

Первый уровень — токенная агрегация. Для каждого чанка выполняется mean pooling скрытых состояний всех токенов с учётом маски внимания. Скрытые состояния поэлементно умножаются на маску внимания, суммируются по оси последовательности и делятся на количество реальных (не паддинговых) токенов. Таким образом, каждый чанк преобразуется в один вектор фиксированной размерности, отражающий агрегированное содержание данного фрагмента документа. Паддинговые токены исключаются из расчёта, что обеспечивает корректность усреднения для чанков разной реальной длины.

Второй уровень — чанковая агрегация. Векторы всех чанков одного документа усредняются с учётом маски числа фрагментов, исключаящей паддинговые чанки из расчёта. Если документ содержит меньше чанков, чем максимум, паддинговые позиции имеют нулевую маску и не влияют на итоговый вектор. Результат — единый документный вектор, представляющий собой взвешенное среднее по всем реальным чанкам. Такая двухуровневая схема выбрана как простой и устойчивый способ агрегации без введения дополнительных обучаемых параметров, что снижает риск переобучения на малой выборке.

Encoder применяется к каждому чанку независимо, но внутри одного общего прохода. Входной тензор с чанками разворачивается в батч последовательностей, обрабатывается encoder'ом параллельно на GPU, а затем результаты приводятся обратно к виду «документ × чанк». Такой способ позволяет одновременно извлекать представления для всех фрагментов документа и не усложнять архитектуру дополнительными связями между чанками.

## 6.11 Классификационная голова и функция потерь

Классификационная голова в chunked-классификаторе состоит из трёх частей: нормализации входного вектора, слоя Dropout и финального линейного слоя. В ранних версиях использовалась более сложная схема с двумя линейными слоями и нелинейностью

GELU между ними. На практике оказалось, что для небольшой выборки и большого числа классов такой вариант быстро уходит в переобучение: модель начинает запоминать обучающие примеры уже на первых эпохах, а качество на валидации либо нестабильно, либо даже падает. Упрощение головы до последовательности LayerNorm → Dropout → Linear сделало обучение заметно более устойчивым и позволило получить лучшую итоговую метрику. Для задач, где данных мало, а модель и так достаточно мощная, избыточно сложная голова чаще мешает, чем помогает: она даёт лишнюю свободу подогнаться под обучающую выборку вместо того, чтобы обобщать.

Нормализация перед классификатором стабилизирует масштаб входного вектора. Это особенно важно в нашем случае, потому что документный эмбединг получается после нескольких этапов агрегации по чанкам, а сами документы могут сильно различаться по длине и структуре. LayerNorm сглаживает эти различия и делает обучение более предсказуемым. Dropout, в свою очередь, уменьшает зависимость предсказаний от отдельных компонент вектора: на каждой итерации часть признаков случайно обнуляется, и модель вынуждена опираться на распределённое представление, а не на единичные «сильные» признаки.

В качестве функции потерь используется кросс-энтропия, но с двумя важными дополнениями: учётом весов классов и сглаживанием меток. Веса классов рассчитываются на основе частот в обучающей выборке: более редкие категории получают больший вес по сравнению с частыми. Это позволяет увеличить их вклад в общий градиент и снизить вероятность того, что модель будет игнорировать их на фоне доминирующих классов. Сглаживание меток, в свою очередь, делает целевое распределение менее жёстким: вместо строгого распределения с единицей для правильного класса используется слегка размытая цель. Такой подход снижает излишнюю уверенность модели в предсказаниях и помогает избежать переобучения, особенно в условиях наличия шумных или неидеальных аугментированных данных.

## 6.12 Параметры обучения chunked-классификатора

Обучение chunked-классификатора должно учитывать сразу несколько факторов: использование крупного encoder, работу с длинными документами, а также небольшой и несбалансированный обучающий набор. Настройки обучения подбирались с оглядкой на ограничения по видеопамяти.



Поскольку каждый документ разбивается на несколько чанков, один объект фактически преобразуется в мини-батч. Это серьёзно увеличивает потребление видеопамяти: в одном шаге модель видит сразу несколько длинных фрагментов текста. Поэтому «физический» размер батча в терминах документов устанавливается равным одному. Чтобы при этом не обучаться на совсем маленьком батче, используется накопление градиентов: модель делает несколько шагов вперёд по одному документу за раз, суммирует градиенты, а обновление параметров выполняет только после заданного числа таких шагов. Эффективный размер батча в этом режиме равен количеству документов, прошедших через модель между обновлениями. Это позволяет вести обучение так, как будто батч крупнее, не выходя за пределы доступной памяти.

Параметры encoder и классификационной головы разделены на отдельные группы, но не ради разных скоростей обучения, а для разного режима регуляризации. Скорости обучения в конечной конфигурации делаются одинаковыми, а вот weight decay применяется только к весам линейных слоёв. Параметры нормализации и смещения (bias) от него исключаются, так как они важны для стабильности обучения и не должны дополнительно штрафиться. Такой раздельный режим позволяет аккуратно контролировать степень регуляризации без подстройки десятков гиперпараметров.

Для управления скоростью обучения используется планировщик с cosine-затуханием и фазой разогрева (warmup) в начале обучения. На начальном этапе скорость обучения постепенно увеличивается от нуля до заданного значения, что позволяет избежать резких обновлений параметров в момент, когда классификационная голова ещё неустойчива и может порождать шумные градиенты. После этапа разогрева скорость обучения плавно уменьшается по cosine-кривой к концу обучения. Такой режим обеспечивает более стабильную сходимость и помогает модели достигать лучшего качества.

Дополнительно применяется обрезка градиента по глобальной норме. Если суммарная норма превышает заданный порог, градиенты масштабируются так, чтобы оставаться в допустимых пределах. Это защищает обучение от редких, но критичных ситуаций с аномально большими градиентами, которые могут возникать, например, на нетипичных документах и приводить к слишком резким обновлениям параметров.

Также используется ранняя остановка обучения. Модель отслеживает значение целевой метрики на валидационной выборке и прекращает обучение, если она не улучшается на протяжении нескольких эпох. Это особенно важно при малой обучающей

выборке: после определённого момента модель начинает подстраиваться под специфические детали train-набора, а качество на валидации уже не растёт. В процессе обучения сохраняются веса, соответствующие лучшему значению метрики на валидации, и именно они используются для финальной оценки, а не параметры последней эпохи.

### 6.13 Приёмы регуляризации и стабилизации обучения

При относительно небольшом количестве обучающих примеров и большом числе параметров риск переобучения для chunked-классификатора очень высок. Поэтому в модели одновременно используются несколько приёмов регуляризации, каждый из которых закрывает свою часть проблем.

Dropout в классификационной голове помогает избежать ситуации, когда решение основано на узком наборе признаков. На каждой итерации часть компонент документного вектора искусственно обнуляется, и модель вынуждена опираться на более широкое распределение информации. Это снижает склонность к запоминанию отдельных паттернов из обучающей выборки и улучшает обобщающую способность на новых письмах.

Сглаживание меток не позволяет выходным вероятностям упираться в идеальные «один» и «ноль». Целевой вектор немного размывается: небольшая часть вероятности распределяется по остальным классам. В результате модель не стремится быть чрезмерно уверенной в каждом предсказании и менее склонна подстраиваться под отдельные шумные примеры. Это особенно актуально в присутствии аугментированных данных: такие примеры не всегда идеально отражают реальные письма, и слишком жёсткая подгонка под них может ухудшить поведение модели на настоящих документах.

Использование весов классов в функции потерь помогает сбалансировать вклад разных категорий. Без этого градиент в основном формировался бы за счёт крупнейших классов, а редкие категории оказывали бы очень слабое влияние на обучение. Веса классов, зависящие от их частот, компенсируют этот эффект: ошибки по редким классам становятся «дороже», и модель имеет стимул лучше их различать. В сочетании с аугментацией, которая увеличивает число примеров в малых классах, это позволяет выровнять условия обучения по всем категориям, а не только по наиболее массовым.

Отдельно используется gradient checkpointing, который отвечает не за регуляризацию, а за возможность вообще обучать такую модель в доступных ресурсах. Вместо того чтобы хранить все промежуточные активации для вычисления градиентов, модель запоминает только часть контрольных точек, а остальное пересчитывает при

обратном проходе. Это уменьшает потребление видеопамати и даёт возможность обучать ruRoberta-large с несколькими чанками на документ на GPU стандартного объёма. Цена за это — небольшое увеличение времени одной итерации, но без такой техники обучение полностью было бы невозможно.

## 7 Композитные и эмбединговые представления текста

### 7.1 Варианты BERT-эмбедингов: предобученные и дообученные

Исследуются два варианта построения эмбедингов, различающиеся наличием адаптации encoder к целевым классам.

Эмбединги предобученной модели строятся прямым применением ruRoberta-large без дополнительного дообучения. В этом случае модель используется как готовый языковой encoder, обученный на широком корпусе русскоязычных текстов. Такой вариант позволяет проверить, насколько общие знания о языке, зафиксированные в предобученной модели, применимы к предметной области деловой переписки без явной адаптации к структуре классов. Преимущество — простота и воспроизводимость: эмбединги вычисляются без этапа обучения. Недостаток — отсутствие специализации: модель может хорошо отражать общую семантическую близость, но не обязательно выделяет признаки, критичные для различения корпоративных писем по управленческим блокам. Эти эмбединги служат baseline плотного представления и позволяют оценить «out-of-the-box» качество encoder на целевой задаче.

Эмбединги дообученной модели получаются после адаптации encoder на данных целевой задачи. Цель такого дообучения — сформировать пространство, в котором документы одного класса располагаются ближе друг к другу, чем документы разных классов. Дообучение выполняется по схеме, описанной в разделе 6.4.1: используются пары текстовых фрагментов из разных документов одного класса, а оптимизация проводится с помощью MultipleNegativesRankingLoss. Такая постановка предотвращает обучение на тривиальной близости перекрывающихся фрагментов одного письма и заставляет encoder извлекать признаки, устойчивые на уровне класса. После дообучения encoder применяется для построения эмбедингов тем же способом, что и предобученная версия: разбиение на чанки, независимое кодирование, агрегация и нормализация.

Сравнение этих вариантов позволяет изолированно оценить вклад адаптации encoder к структуре целевых классов. Если дообученные эмбединги значительно превосходят предобученные, это указывает на важность метрического обучения для данной задачи.

### 7.2 Классические классификаторы поверх плотных эмбедингов

После построения BERT-эмбедингов задача классификации сводится к обучению моделей уже не на текстах, а на плотных векторах фиксированной размерности. Каждый

документ представлен одним эмбедингом, и дальше можно применять стандартные алгоритмы машинного обучения. Такой подход позволяет отдельно оценить качество самого признакового пространства: если эмбединги хорошо отражают различия между классами, даже простые линейные модели должны проводить разумную разделяющую границу.

В работе используются LinearSVC и LogisticRegression. Обе модели строят линейные разделители в пространстве эмбедингов, но опираются на разные функции потерь и по-разному интерпретируют результат. LinearSVC реализует SVM с линейным ядром и оптимизируется по hinge-loss, фокусируясь на максимизации зазора между классами. LogisticRegression минимизирует логистическую функцию потерь и возвращает оценку вероятностей по классам, что удобно для анализа уверенности модели и порогового выбора. Сопоставление их результатов показывает, насколько классы линейно делимы в BERT-пространстве и требуется ли более сложная модель.

С практической точки зрения этот подход очень экономичен по вычислительным затратам. Encoder прогоняется по данным один раз, эмбединги сохраняются, а обучение линейных моделей поверх готовых векторов занимает секунды на CPU. Те же самые эмбединги используются в разных ветках экспериментов, поэтому различия в качестве можно напрямую связывать с выбором классификатора, а не с тем, как именно построены признаки.

### **7.3 Методы классификации по косинусному сходству**

Кроме обучения классификаторов, эмбединги используются и в методах прямого сравнения документов по косинусному сходству. В отличие от обучаемых моделей, эти методы не строят явных разделяющих границ, а работают непосредственно с геометрией пространства: насколько близко расположены векторы для разных документов.

Вариант с центроидами устроен так: для каждого класса вычисляется средний вектор по всем его обучающим документам. Для нового письма строится эмбединг, после чего находится класс с максимальным косинусным сходством к своему центроиду. Если эмбединговое пространство организовано хорошо, документы одного класса тяготеют к своему центру, а центроиды разных классов оказываются достаточно разнесены. Ограничение этого подхода в том, что один усреднённый вектор не всегда способен адекватно описать классы с внутренним разнообразием: при наличии нескольких подгрупп

внутри класса центроид может оказаться «где-то посередине» и плохо соответствовать любому реальному документу.

На практике классы оказываются не только мультимодальными, но и зашумлёнными: в обучающей выборке встречаются документы, которые формально приписаны к классу, но по смыслу далеки от его «ядра». Простой средний вектор в такой ситуации систематически смещается в сторону шумовых примеров. Поэтому центроид строится в более устойчивом виде, чем простое усреднение. Сначала для каждого класса вычисляется предварительный средний вектор; затем для каждого обучающего примера считается его косинусное сходство с этим средним; примеры с наименьшим сходством либо полностью исключаются из агрегата, либо получают пониженный вес, после чего средний вектор пересчитывается. Эта процедура может повторяться несколько раз, на каждой итерации опираясь на центроид с предыдущего шага. Таким образом, итоговый центроид класса — это не среднее всех помеченных примеров, а среднее наиболее «центральных» из них, что снижает чувствительность к разметочному шуму и к редким выбросам внутри класса.

Метод ближайшего соседа действует иначе. Для тестового документа ищется обучающий пример с максимальным косинусным сходством, и его метка переносится на новый документ. Здесь сохраняется локальная структура пространства: если внутри класса есть разные кластеры, ближайший сосед, как правило, будет взят из подходящего кластера, даже если глобальный центроид расположен далеко. Цена за это — чувствительность к шуму и ошибочным меткам в обучающей выборке: один нетипичный или неверно размеченный документ может стать «плохим» соседом и привести к неправильной классификации. Дополнительно, по сравнению с центроидами, метод требует считать сходство с большим числом объектов, что заметнее по ресурсам.

В чистом виде метод ближайшего соседа уязвим к локальным выбросам: единственный неверно размеченный близкий обучающий документ полностью определяет ответ. Поэтому в работе используется обобщённый вариант: рассматривается не один ближайший представитель, а несколько ближайших по косинусу. Их сходства преобразуются в распределение вероятностей классов с помощью температурного софтмакса — более близкие соседи получают больший вес, а температура регулирует, насколько резко вес падает при отдалении. После этого вероятности соседей суммируются по классам, и выбирается класс с наибольшим суммарным голосом. Размер окрестности и температура подбираются на основе качества классификации; при достаточно низкой

температуре поведение метода стремится к классическому одному ближайшему соседу, а при высокой — приближается к равноправному голосованию по всем ближайшим точкам.

Метод центроида и метод взвешенного ближайшего соседа имеют противоположные свойства: первый описывает класс одним обобщённым представлением и устойчив к локальному шуму, но плохо работает на мультимодальных классах; второй внимателен к локальной структуре, но чувствителен к выбросам. Поэтому в работе также рассматривается их объединение. Для каждого объекта вычисляется косинусное сходство со всеми центроидами классов и одновременно — взвешенное голосование по ближайшим соседям. Обе эти оценки нормируются и линейно смешиваются с коэффициентом, который определяет, какой из двух подходов в итоговом решении превалирует. При крайних значениях коэффициента метод вырождается в чистый центроидный либо в чистый соседский, а при промежуточных значениях получается компромисс, который в наших экспериментах оказывается заметно лучше каждого из исходных методов по отдельности.

Все три метода используются в работе не как основные решения, а как инструмент анализа эмбедингового пространства. Их поведение помогает понять, насколько компактны классы, есть ли у них внутренние подгруппы и насколько хорошо BERT-эмбединги отделяют разные категории без дополнительного обучаемого классификатора.

#### **7.4 Конкатенация TF-IDF и BERT-векторов**

Разреженные TF-IDF-признаки и плотные BERT-эмбединги отражают разные свойства текста. TF-IDF хорошо фиксирует точные лексические совпадения: конкретные термины, аббревиатуры, внутренние обозначения подразделений, характерные фразы деловой переписки. Если два письма содержат одну и ту же формулировку (например, название проекта или код департамента), TF-IDF зафиксирует это как прямое совпадение соответствующих компонент вектора. BERT-эмбединги, напротив, позволяют учитывать контекст и семантическую близость, различая значения одних и тех же слов в разных окружениях. Они полезны для обобщения по документам с разными формулировками, но схожим содержанием. Поэтому в работе исследуется объединение этих представлений в один композитный вектор.

Конкатенация выполняется на уровне признаков: для каждого документа строится TF-IDF-вектор и BERT-вектор, затем они объединяются в одно общее пространство. В результате классификатор получает одновременно разреженные лексические признаки и

плотные контекстные признаки, что потенциально позволяет использовать преимущества обоих подходов. Для деловых писем это особенно важно: класс может определяться как точными маркерами (название проекта, код подразделения, устойчивые фразы), так и общим содержанием документа (описание действий, запросов, характер обсуждения).

## **7.5 Реализация композитного признакового пространства**

Композитное признаковое пространство строится в несколько этапов. Сначала для каждого документа формируется TF-IDF-представление. Word-компонента включает униграммы и биграммы слов, char-компонента — символьные n-граммы длины от трёх до пяти. Эти признаки объединяются в разреженный вектор, фиксирующий лексические паттерны текста.

Параллельно для каждого документа формируется BERT-эмбединг с использованием стратегии разбиения на чанки, описанной ранее. В результате получается вектор фиксированной размерности, соответствующий скрытому пространству модели ruRoberta-large.

Перед объединением оба типа признаков нормализуются независимо: TF-IDF- и BERT-векторы приводятся к единичной норме. Это необходимо из-за различий в их природе и масштабе значений. При этом для baseline-эмбедингов, полученных простым усреднением скрытых состояний модели, перед нормировкой к BERT-блоку дополнительно применяется покоординатное стандартное масштабирование, тогда как для дообученного кастомного эмбеддера это масштабирование отключено — его представления уже выровнены по норме за счёт обучения с triplet- и multiple-negatives-ranking-функциями потерь. Затем BERT-компонента дополнительно масштабируется с помощью весового коэффициента, что позволяет регулировать её вклад в итоговое представление. Значение коэффициента подбирается эмпирически, чтобы компенсировать различие размерностей и обеспечить сопоставимое влияние плотных и разреженных признаков. После этого выполняются конкатенация векторов и финальная L2-нормализация объединённого представления.

При практическом использовании композитного пространства обнаружилось, что классические линейные модели и многослойная сеть предъявляют разные требования к норме итогового вектора. Классические модели опираются на абсолютные значения признаков и на регуляризацию, заданную в их пространстве, поэтому им важно, чтобы внутренний баланс между TF-IDF-блоком и BERT-блоком, задаваемый весовым



коэффициентом, сохранялся в неизменном виде. Многослойная сеть, наоборот, исходно настраивалась на признаки с фиксированной нормой, и сильно меняющаяся длина вектора между объектами для неё означает скачок эффективной скорости обучения и потерю стабильности. Поэтому в реализации одновременно хранятся две версии композитного представления — с финальной L2-нормировкой склеенного вектора и без неё. Первая используется многослойной сетью, вторая — классическими моделями. На уровне отдельных блоков (TF-IDF и BERT по отдельности) обе версии идентичны: они L2-нормированы и масштабированы весовым коэффициентом. Различие проявляется только на этапе финальной агрегации.

Полученное композитное пространство используется как вход для последующих классификаторов. Его основная идея — не выбирать между классическим и нейросетевым представлением, а объединить их в одном векторе документа, позволяя модели использовать оба типа информации одновременно.

## **7.6 Классические модели и MLP поверх композитных векторов**

После построения композитных векторов поверх них обучаются классификационные модели двух типов: классические линейные модели и MLP-классификатор. Это позволяет проверить, достаточно ли линейной разделимости объединённого пространства или требуется нелинейное преобразование признаков.

Классические модели — LinearSVC и LogisticRegression — применяются к композитным векторам так же, как ранее к чистым TF-IDF- и BERT-представлениям. Это позволяет посмотреть, как меняется поведение уже знакомых линейных классификаторов, когда они получают на вход объединённое пространство признаков, и даёт возможность понять, даёт ли сама комбинация признаков выигрыш по сравнению с отдельными ветками.

Дополнительно используется MLP-классификатор. В отличие от линейных моделей, он может учитывать нелинейные взаимодействия между компонентами композитного вектора. Это особенно интересно в ситуации, когда TF-IDF-часть отражает точные лексические маркеры, а BERT-часть — контекст и общий смысл текста. Нелинейная модель потенциально способна «склеивать» эти два источника информации и использовать их совместно, а не по отдельности.

Многослойная сеть в эксперименте имеет вид входной линейной проекции в скрытое пространство фиксированной размерности, нескольких остаточных блоков с послойной нормализацией, нелинейностью и dropout, и линейной классификационной головы.

Архитектура с остаточными связями выбрана для того, чтобы при сравнительно небольшом объёме обучающих данных и большом числе классов сеть не сваливалась в коллапс на доминирующие классы. В обучении используются три приёма, направленные именно на проблему несбалансированности классов: взвешивание классов в функции потерь обратно пропорционально частоте, *focal-loss* для дополнительного смещения внимания в сторону «трудных» примеров и *mixup* в пространстве признаков, который синтетически увеличивает вариативность обучающей выборки в окрестности классовых границ. Расписание скорости обучения — косинусное без разогрева, ранняя остановка по макроусреднённой *f1*-мере на тестовой выборке. Подбор гиперпараметров проводился по нескольким профилям с разными значениями скрытой размерности и количества блоков; в итоговом сравнении используется единый профиль, обеспечивающий наилучшее качество и одинаковые условия для обеих версий эмбеддингов.

MLP обучается поверх уже сформированных композитных векторов, то есть этап построения признаков отделён от этапа классификации. Благодаря этому все классификаторы — линейные и нелинейный — сравниваются на одном и том же входном представлении. Это делает интерпретацию результатов прозрачнее: изменения качества можно связывать либо с типом признаков, либо с выбором модели, а не с скрытыми отличиями в подготовке данных.

### **7.7 Роль композитных признаков в общей схеме**

Композитная ветка объединяет результаты двух линий исследования: классического TF-IDF-представления и контекстных BERT-эмбеддингов. В ней проверяется, что будет, если рассматривать их не по отдельности, а совместно — в виде общего признакового пространства, поверх которого можно обучать разные классификаторы. Такой подход позволяет оценить, дают ли разреженные и плотные признаки в сумме больше информации о классе, чем каждый из них по отдельности.

Основная идея состоит в том, что разные типы признаков отвечают за разные стороны задачи. TF-IDF хорошо работает там, где класс «зашит» в устойчивых формулировках, терминах или названиях. BERT-эмбеддинги, напротив, полезны, когда важен общий контекст письма и типичные описания ситуаций, а не конкретные слова. Для длинных деловых писем это особенно актуально: принадлежность к классу может определяться и прямыми упоминаниями проектов или подразделений, и общим характером описываемых действий. Композитные признаки как раз и проверяют гипотезу о том, что

совместное использование этих двух источников информации позволяет лучше учесть оба аспекта одновременно.

## 8 Программная реализация пайплайна

### 8.1 Структура программного решения

Программная часть работы оформлена в виде модульного исследовательского пайплайна. Каждый крупный этап — подготовка данных, очистка, аугментация, суммаризация, построение признаков, обучение и оценка — вынесен в отдельный модуль или группу модулей. Такой подход упрощает повторный запуск экспериментов, позволяет менять отдельные компоненты независимо и не смешивать в одном месте логику предобработки, обучения и анализа результатов. Разные версии экспериментов фиксируются в системе контроля версий вместе с полученными метриками.

В репозитории выделены несколько основных направлений: подготовка исходного датасета, очистка и нормализация текстов, аугментация, суммаризация, классические baseline-модели, работа с BERT-эмбедингами, end-to-end BERT-классификация, классификация по косинусному сходству и построение гибридных признаков. Такая организация напрямую отражает логику исследования: сначала формируется подготовленный корпус данных, после чего на его основе строятся различные представления и обучаются соответствующие модели.

Ключевым начальным этапом является подготовка данных из исходного файла. Для этого используется отдельный скрипт, выполняющий очистку, формирование структурированного датафрейма и создание разбиения на обучающую и тестовую выборки. Далее все ветки пайплайна работают уже с подготовленными данными — например, файлами `train.csv` и `test.csv`, а также их аугментированными и суммаризованными версиями. Это позволяет проводить новые эксперименты без повторной обработки исходных сырых данных.

С инженерной точки зрения решение организовано так, чтобы отдельные эксперименты можно было запускать независимо, без полной пересборки всего проекта. Например, модели на основе TF-IDF, BERT-классификация и гибридные подходы реализованы в отдельных частях репозитория, но используют общий подготовленный корпус. Это упрощает сопровождение кода и делает возможным постепенное расширение экспериментальной части.

## 8.2 Модуль предобработки данных

Модуль предобработки предназначен для преобразования исходных писем в очищенный и нормализованный текстовый корпус. Основная логика подготовки данных реализована в `prepare_dataset.py` и вспомогательных скриптах каталога `utils/`. Этот этап выполняется до обучения моделей и является общим для всех последующих экспериментальных веток.

В процессе предобработки выполняется чтение исходного набора данных, удаление или нормализация технических фрагментов, проверка структуры данных и сохранение очищенного датафрейма. Для очистки используются регулярные выражения и эвристики, направленные на обработку служебных блоков, повторяющихся фрагментов, списков, реквизитов и других элементов, которые могут создавать шум в признаковом пространстве.

Отдельная часть предобработки связана с формированием `train/test`-разбиения. После очистки данные делятся на обучающую и тестовую части с сохранением всех классов. Это разбиение далее используется во всех экспериментах, что обеспечивает сопоставимость результатов. Тестовая часть не используется при аугментации и обучении моделей.

Результатом работы модуля предобработки являются подготовленные файлы данных с колонками `text` и `label`. Такой простой формат удобен для дальнейшей работы: классические модели, BERT-пайплайны, аугментация, суммаризация и гибридные подходы могут использовать одинаковую структуру входных данных.

## 8.3 Модуль аугментации текстов

Модуль аугментации реализован в каталоге `augmentation/` и разделён на несколько логических частей. Общие функции вынесены в подкаталог `common/`, а конкретные методы аугментации расположены в отдельных ветках `paraphrase/` и `backtranslate/`. Такое разделение позволяет использовать общую инфраструктуру обработки длинных текстов для разных способов генерации новых примеров.

Общая логика аугментации включает маскирование значимых сущностей, нормализацию текста, разбиение документа на предложения и фрагменты, генерацию нескольких кандидатов, фильтрацию по качеству и последующую сборку полного документа. Такой подход необходим из-за большой длины деловых писем: генерация нового текста целиком была бы менее устойчивой и сильнее зависела бы от ограничений используемых моделей.

В ветке перефразирования используется генеративная ruT5-модель, создающая альтернативные формулировки русскоязычных фрагментов. В ветке обратного перевода применяются модели машинного перевода семейства Helsinki-NLP/OPUS-MT, где русский текст переводится на промежуточный язык и обратно. Для обоих методов реализована фильтрация кандидатов по косинусному сходству и дополнительная постобработка сгенерированного текста.

Отдельно реализована логика доведения редких классов до заданного порогового размера. Генерация новых примеров выполняется преимущественно для тех классов, где исходных документов недостаточно. Итогом работы модуля становятся отдельные аугментированные наборы, например `train_paraphrase.csv`, `train_backtranslate.csv`, а также объединённый `train_augmented.csv`.

## 8.4 Модуль построения векторных представлений

Построение векторных представлений реализовано в нескольких частях проекта, поскольку в работе используются разные типы признаков. Классические разреженные признаки строятся в `baseline`-ветке на основе `TfidfVectorizer`. Для этого используются `word n`-граммы, `char n`-граммы и их объединение через `FeatureUnion`.

Для BERT-представлений используется каталог `bert_embeddings/`. В нём реализовано построение плотных эмбеддингов документов на основе `ai-forever/ruRoberta-large`. Так как письма могут быть длинными, используется отдельный механизм обработки длинных текстов: документ разбивается на чанки, каждый чанк кодируется моделью, после чего представления агрегируются в единый вектор документа.

Отдельная часть построения признаков относится к композитным векторам. Она реализована в каталоге `hybrid/`. В этой ветке для одного и того же документа строятся TF-IDF-признаки и BERT-эмбеддинг, после чего они объединяются в общее признаковое пространство. Перед объединением выполняется нормализация, а вклад BERT-компоненты может дополнительно масштабироваться.

Такая организация позволяет использовать разные представления независимо друг от друга. Один и тот же подготовленный текст может быть преобразован в разреженный TF-IDF-вектор, плотный BERT-вектор или композитный вектор. Это обеспечивает единую основу для сравнения разных подходов к классификации.

## 8.5 Модуль обучения моделей

Обучение моделей также разделено на несколько независимых веток. Классические модели реализованы в `baseline.py` и скриптах каталога `baseline_experiments/`. В этих экспериментах проверяются разные варианты TF-IDF-признаков и классификаторов, включая `LinearSVC` и `LogisticRegression`.

BERT-классификация реализована в каталоге `bert_classification/`. В нём выделены отдельные файлы для конфигурации, подготовки данных, модели, обучения, инференса и расчёта метрик. Основной классификатор построен на основе `ai-forever/ruRoberta-large` и поддерживает обработку длинных документов через разбиение на чанки. Для обучения используется MLP-голова, веса классов, `label smoothing` и разные `learning rate` для `encoder` и классификационной головы.

Ветка эмбедингов и Sentence-BERT-подхода реализована в `bert_embeddings/`. Она отвечает за обучение модели представлений и последующее получение эмбедингов документов. Эти эмбединги далее могут использоваться в классических классификаторах, методах косинусной классификации и гибридных моделях.

Методы классификации по косинусному сходству вынесены в каталог `cosine_similarity_classification/`. Там реализованы подходы на основе центроидов классов и ближайшего соседа. Гибридные классификаторы расположены в `hybrid/`, где поверх объединённых TF-IDF и BERT-векторов обучаются классические модели и MLP-классификатор.

## 8.6 Воспроизводимость экспериментов

Для исследовательской работы важна воспроизводимость экспериментов, поэтому в реализации фиксируются основные параметры подготовки данных, обучения и оценки моделей. Train/test-разбиение создаётся с фиксированным `random_state=42`, что позволяет повторно получать одинаковое разделение данных. Во многих ветках также используется фиксированный `seed` для генераторов случайных чисел.

В пайплайне сохраняются промежуточные результаты: очищенный датафрейм, `train/test`-файлы, отдельные аугментированные наборы, объединённый аугментированный набор, суммаризованные варианты и построенные признаки. Это позволяет не пересчитывать каждый раз все этапы с нуля и запускать отдельные эксперименты независимо.

Конфигурационные параметры вынесены в отдельные файлы или явно заданы в скриптах. Например, для BERT-классификатора отдельно задаются параметры модели, данных и обучения: максимальная длина чанка, stride, число чанков, learning rate, dropout, label smoothing, число эпох и параметры ранней остановки. В классических моделях зафиксированы параметры TF-IDF-векторизации и классификаторов.

Дополнительно в репозитории хранится сводка экспериментальных результатов. Это позволяет сопоставлять разные ветки моделирования и отслеживать, на каких данных и с какими настройками были получены результаты. Такая организация делает пайплайн не только исследовательским, но и воспроизводимым: каждый этап можно повторить или модифицировать без нарушения общей структуры проекта.

## **8.7 Ограничения реализации и требования к вычислительным ресурсам**

Разработанный пайплайн включает как лёгкие классические модели, так и более ресурсоёмкие BERT-подходы. Классическая ветка TF-IDF + LinearSVC или LogisticRegression может обучаться на CPU и не требует значительного объёма видеопамати. Это делает её удобной для быстрых экспериментов, проверки данных и получения базовой линии качества.

BERT-подходы предъявляют более высокие требования к вычислительным ресурсам. Особенно это актуально для модели ai-forever/ruRoberta-large, применяемой для классификации, построения эмбеддингов и обработки длинных документов. При использовании чанкинга один документ разбивается на несколько фрагментов, что приводит к увеличению нагрузки на память и времени обучения.

Для снижения потребления памяти используются небольшой размер батча и накопление градиентов. В основном BERT-классификаторе применяется конфигурация с batch size, равным одному, и накоплением градиентов на нескольких шагах, что позволяет обучать модель в условиях ограниченной видеопамати. Дополнительно используются такие техники, как gradient checkpointing, ограничение числа чанков на документ и ранняя остановка обучения.

Дополнительное ограничение связано со временем генерации аугментированных данных. Методы перефразирования и обратного перевода применяются к отдельным фрагментам текста, при этом для каждого из них может создаваться несколько вариантов. Это улучшает качество итогового отбора, но увеличивает общее время подготовки данных.



Поэтому аугментация реализована как отдельный этап, результаты которого сохраняются в файлы и затем переиспользуются.

Таким образом, программная реализация поддерживает несколько уровней вычислительной сложности. Быстрые классические модели позволяют оперативно проверять гипотезы, а более тяжёлые BERT- и hybrid-подходы используются для исследования контекстных и композитных представлений текста. Такая структура позволяет работать с задачей в условиях ограниченных ресурсов и при этом сохранять возможность расширения пайплайна.

## 9 Экспериментальное исследование и сравнение результатов

### 9.1 Экспериментальный протокол оценки качества

Экспериментальное исследование проводилось на фиксированном train/test-разбиении, сформированном после очистки и нормализации исходного датасета. Обучающая выборка использовалась для построения признаков, обучения моделей, аугментации и формирования дополнительных вариантов данных. Тестовая выборка оставалась неизменной и применялась только для итоговой оценки качества.

В рамках экспериментов сравнивались несколько групп подходов. Первая группа включала классический baseline на основе TF-IDF-признаков и линейного классификатора. Вторая группа была основана на BERT-классификации с использованием ruRoberta-large. Отдельно рассматривались подходы на основе косинусного сходства в пространстве эмбедингов и гибридные модели, объединяющие TF-IDF- и BERT-представления.

Для проверки влияния состава обучающих данных использовались четыре варианта train-набора: исходный текст, аугментированный текст, суммаризованный текст и комбинированный вариант, где исходный документ объединялся с его кратким содержанием. Такой протокол позволил сравнить не только модели, но и способы представления исходного письма.

Все результаты далее приводятся в едином формате: модель обучается на выбранном варианте train-набора и оценивается на одной и той же тестовой выборке. Это делает сравнение между подходами корректным и позволяет анализировать влияние отдельных компонентов пайплайна: очистки, аугментации, суммаризации, выбора модели и способа построения признаков.

### 9.2 Используемые метрики качества классификации

Для оценки качества использовались метрики, подходящие для многоклассовых задач с несбалансированными классами. Основной акцент делается на balanced accuracy и macro F1, так как обе метрики учитывают вклад каждого класса и не позволяют крупным категориям «перетянуть» итоговый результат за счёт своего количества. Обычная accuracy в такой постановке может быть завышена, если модель хорошо работает только на самых частых классах, поэтому она не рассматривается как основная.

Balanced accuracy вычисляется как средняя полнота по всем классам и показывает, насколько равномерно модель распознаёт каждую категорию, включая редкие. Если модель

уверенно классифицирует только массовые классы и игнорирует малочисленные, *balanced accuracy* окажется заметно ниже обычной *accuracy* и лучше отразит реальную устойчивость модели. *Macro F1* вносит равный вклад каждого класса в итоговый показатель, объединяя информацию о точности и полноте. В данной работе *macro F1* используется как одна из ключевых метрик именно потому, что важно оценить не только общий процент правильных ответов, но и способность модели адекватно работать со всей совокупностью классов, в том числе редкими.

### 9.3 Влияние аугментации на TF-IDF и BERT-модели

Первым направлением анализа было сравнение качества моделей на исходной и аугментированной обучающей выборке. Аугментация применялась в первую очередь для увеличения числа примеров в редких классах и выравнивания *train*-набора, поэтому её влияние рассматривалось отдельно для классического TF-IDF-baseline и для BERT-классификаторов. Это позволило понять, как один и тот же приём воздействует на разные типы признакового пространства.

Для TF-IDF-baseline качество на аугментированном наборе оказалось ниже, чем на исходном: и *balanced accuracy*, и *macro F1* лучше на «чистом» *train*. Это можно объяснить тем, что исходная обучающая выборка уже представляет собой отфильтрованный, содержательный корпус, где каждый документ даёт понятный сигнал в TF-IDF-пространстве. Добавление синтетических примеров хоть и расширяет лексическое разнообразие, но одновременно «размывает» границы между классами: тексты с промежуточными формулировками оказываются между кластерами и усложняют задачу линейному классификатору.

Для BERT-классификаторов наблюдается обратная картина: во всех рассмотренных конфигурациях (ruBERT и ruRoberta, варианты с CLS, mean pooling и chunkmean) обучение на аугментированном *train*-наборе приводит к росту и *balanced accuracy*, и *macro F1*. Это согласуется с ожиданием, что нейросетевые модели сильнее выигрывают от разнообразия формулировок и способны извлекать полезный смысл из синтетических примеров, улучшая обобщение на тестовой выборке. В итоге аугментация выступает не как универсальное средство «поднять метрики», а как приём, эффективность которого зависит от типа признаков и модели.

Таблица 9.1 — Влияние аугментации на TF-IDF baseline и BERT-классификаторы

| Подход                        | Обучающие данные | Balanced accuracy | Macro F1 |
|-------------------------------|------------------|-------------------|----------|
| TF-IDF baseline               | train            | 0.534             | 0.543    |
| TF-IDF baseline               | train_augmented  | 0.489             | 0.486    |
| rubert-base-cased CLS         | train            | 0.359             | 0.347    |
| rubert-base-cased CLS         | train_augmented  | 0.433             | 0.431    |
| rubert-base-cased MeanPooling | train            | 0.406             | 0.408    |
| rubert-base-cased MeanPooling | train_augmented  | 0.453             | 0.448    |
| ruRoberta-large CLS           | train            | 0.388             | 0.377    |
| ruRoberta-large CLS           | train_augmented  | 0.507             | 0.488    |
| ruRoberta-large MeanPooling   | train            | 0.396             | 0.394    |
| ruRoberta-large MeanPooling   | train_augmented  | 0.479             | 0.469    |
| ruRoberta-large chunkmean     | train            | 0.421             | 0.393    |
| ruRoberta-large chunkmean     | train_augmented  | 0.500             | 0.504    |

Таким образом, аугментация не является универсальным способом улучшения любого классификатора. Она улучшает качество BERT-моделей, но ухудшает TF-IDF baseline, поэтому её использование целесообразно прежде всего в нейросетевой ветке.

#### 9.4 Влияние суммаризации на представление текста

В отдельной серии экспериментов анализировалось, как суммаризация влияет на качество классификации при разных вариантах представления документа: исходный текст, только суммаризованный текст и комбинированное представление original+summary.

Таблица 9.2 — Влияние суммаризации на TF-IDF baseline

| Обучающие данные                      | Balanced accuracy | Macro F1 |
|---------------------------------------|-------------------|----------|
| train_summarized                      | 0.366             | 0.378    |
| train_original_plus_summary           | 0.482             | 0.497    |
| train_augmented_summarized            | 0.384             | 0.395    |
| train_augmented_original_plus_summary | 0.467             | 0.475    |

Для TF-IDF baseline все варианты, в которых исходный текст заменяется или дополняется суммаризацией, дают более низкие метрики по сравнению с обучением на

чистом исходном train-наборе. При этом внутри самих суммаризованных конфигураций комбинированное представление original+summary заметно лучше, чем обучение только на summary: и для неаугментированных данных (0.482 против 0.366 по balanced accuracy), и для аугментированных (0.467 против 0.384). Это говорит о том, что baseline нуждается в полноте исходного текста и теряет важные признаки при переходе к короткому резюме, но сочетание оригинала и краткого содержания частично компенсирует эту потерю.

Таблица 9.3 — Суммаризированный датасет на BERT-классификатор

| Подход                           | Обучающие данные           | Balanced accuracy | Macro F1 |
|----------------------------------|----------------------------|-------------------|----------|
| rubert-base-cased<br>CLS         | train_summarized           | 0.193             | 0.183    |
| rubert-base-cased<br>MeanPooling | train_summarized           | 0.242             | 0.240    |
| ruRoberta-large<br>CLS           | train_summarized           | 0.345             | 0.325    |
| ruRoberta-large<br>MeanPooling   | train_summarized           | 0.333             | 0.326    |
| rubert-base-cased<br>CLS         | train_augmented_summarized | 0.326             | 0.291    |
| rubert-base-cased<br>MeanPooling | train_augmented_summarized | 0.270             | 0.271    |
| ruRoberta-large<br>CLS           | train_augmented_summarized | 0.379             | 0.383    |
| ruRoberta-large<br>MeanPooling   | train_augmented_summarized | 0.411             | 0.412    |

Для BERT-классификаторов обучение только на суммаризованных текстах также приводит к ухудшению качества по сравнению с исходными письмами. На чистом train\_summarized и ruBERT, и ruRoberta показывают заметно более низкие значения balanced accuracy и macro F1, чем при обучении на полном тексте. Подключение аугментации к суммаризованным данным улучшает метрики, особенно для ruRoberta, но даже в этом случае результаты остаются ниже, чем у моделей, обученных на полном тексте с аугментацией. В совокупности это показывает, что для текущей задачи суммаризация в виде самостоятельной замены исходного письма не даёт выигрыша ни для TF-IDF baseline,

ни для BERT-классификаторов, а наилучшее качество достигается при работе с максимально богатым по содержанию представлением письма.

Таблица 9.4 — Комбинированный датасет на BERT-классификатор

| Подход                        | Обучающие данные                      | Balanced accuracy | Macro F1 |
|-------------------------------|---------------------------------------|-------------------|----------|
| rubert-base-cased CLS         | train_original_plus_summary           | 0.363             | 0.358    |
| rubert-base-cased MeanPooling | train_original_plus_summary           | 0.375             | 0.386    |
| ruRoberta-large CLS           | train_original_plus_summary           | 0.383             | 0.381    |
| ruRoberta-large MeanPooling   | train_original_plus_summary           | 0.444             | 0.435    |
| rubert-base-cased CLS         | train_augmented_original_plus_summary | 0.420             | 0.423    |
| rubert-base-cased MeanPooling | train_augmented_original_plus_summary | 0.461             | 0.452    |
| ruRoberta-large CLS           | train_augmented_original_plus_summary | 0.478             | 0.459    |
| ruRoberta-large MeanPooling   | train_augmented_original_plus_summary | 0.511             | 0.498    |

Для BERT-классификаторов комбинированное представление original+summary оказывается промежуточным по качеству между обучением только на полном тексте и обучением только на суммаризованных письмах. Без аугментации метрики для train\_original\_plus\_summary заметно ниже, чем для исходного train из предыдущих экспериментов, но выше, чем для train\_summarized. Подключение аугментации к комбинированному датасету стабильно улучшает результаты для всех вариантов BERT: на train\_augmented\_original\_plus\_summary и balanced accuracy, и macro F1 возрастают, а

лучшая конфигурация (ruRoberta-large с mean pooling) достигает 0.511 по balanced accuracy и 0.498 по macro F1. Это уже сопоставимо с лучшими значениями для моделей, обученных на полном тексте с аугментацией, что показывает, что связка «исходное письмо + краткое резюме» в сочетании с аугментацией даёт BERT-классификатору дополнительный полезный сигнал и может использоваться как конкурентоспособный вариант представления документа.

## 9.5 Косинусные методы

При выборе оптимальной конфигурации варьировались параметры. Для центроидного метода — стратегия построения опорного вектора: жёсткое отбрасывание определённой доли наиболее удалённых примеров либо мягкое перевзвешивание всех примеров пропорционально их близости к предварительной оценке центроида, с настройкой соответствующих коэффициентов. Для метода ближайших соседей — число рассматриваемых соседей и параметр резкости голосования, определяющий относительный вклад более близких и более далёких соседей в итоговое решение. Для ансамблевого варианта — доля участия центроидного и k-NN решений в финальном предсказании.

Таблица 9.5 — Сравнение методов косинусного расстояния на эмбединговом представлении текста

| Эмбединги | Обучающие данные | Метод      | Balanced accuracy | Macro F1 |
|-----------|------------------|------------|-------------------|----------|
| default   | train            | Centroid   | 0.134             | 0.109    |
| default   | train            | Nearest    | 0.198             | 0.187    |
| default   | train            | CentroidNN | 0.171             | 0.168    |
| default   | train_augmented  | Centroid   | 0.099             | 0.096    |
| default   | train_augmented  | Nearest    | 0.197             | 0.186    |
| default   | train_augmented  | CentroidNN | 0.168             | 0.170    |
| custom    | train            | Centroid   | 0.578             | 0.508    |
| custom    | train            | Nearest    | 0.504             | 0.504    |
| custom    | train            | CentroidNN | 0.503             | 0.501    |
| custom    | train_augmented  | Centroid   | 0.448             | 0.461    |
| custom    | train_augmented  | Nearest    | 0.443             | 0.458    |
| custom    | train_augmented  | CentroidNN | 0.442             | 0.456    |

На baseline-эмбедингах все три косинусных метода фактически работают на уровне близком к случайному: macro F1 не поднимается выше 0.19, причём центроидный вариант оказывается самым слабым. Аугментация обучающей выборки ситуацию не улучшает и местами даже ухудшает, что показывает: в пространстве baseline-эмбедингов классы разделены слишком плохо, чтобы простое сравнение по косинусу могло давать надёжную классификацию.

После дообучения эмбеддера всё меняется: на исходном train все три метода выходят на macro F1 около 0.50–0.51, а центроидный классификатор даёт лучшую balanced accuracy. Это означает, что дообучение «перестраивает» пространство так, что документы одного класса начинают собираться ближе друг к другу, а разные классы — заметно разноситься. При этом аугментация для косинусных методов скорее вредна: синтетические примеры размывают границы между классами, а именно эти границы являются главным сигналом для метрических подходов; поэтому в данной задаче ключевым условием их применимости является дообучение эмбеддера, а среди вариантов наиболее устойчивым выглядит центроидный метод.

## **9.6 Сравнение классических, нейросетевых и композитных представлений**

Hybrid-модели в этой главе нужны не «ради ещё одной модели», а как способ проверить сами представления. TF-IDF хорошо ловит точные маркеры (термины, аббревиатуры, названия, устойчивые обороты), а BERT-эмбединги добавляют контекст и смысловую близость. Поэтому композит (TF-IDF + BERT) позволяет увидеть, дают ли эти источники информации прирост вместе и насколько хорошо классы «расходятся» в общем векторном пространстве.

### **9.6.1 MLP-классификатор**

MLP использовался как нелинейный классификатор поверх композитных признаков (TF-IDF по словам и символьным n-граммам + BERT-эмбединги) и реализован как полносвязная сеть с residual-связями. При настройке варьировались архитектура и обучение (размер скрытого слоя, глубина, dropout, скорость обучения и расписание, размер мини-батча), параметры функции потерь и компенсации дисбаланса (class weights, label smoothing, фокусирующая функция), микс в скрытом пространстве, а также weight decay и ранняя остановка. Из-за стохастичности обучения каждая конфигурация запускалась трижды; дальше сравнивались средние значения и разброс.



Таблица 9.6 — все mlp запуски

| Эмбединги | Обучающие данные | Запуск | Balanced accuracy | Macro F1 |
|-----------|------------------|--------|-------------------|----------|
| default   | train            | 1      | 0.527             | 0.517    |
| default   | train            | 2      | 0.493             | 0.512    |
| default   | train            | 3      | 0.542             | 0.513    |
| default   | train_augmented  | 1      | 0.404             | 0.438    |
| default   | train_augmented  | 2      | 0.447             | 0.484    |
| default   | train_augmented  | 3      | 0.490             | 0.505    |
| custom    | train            | 1      | 0.498             | 0.543    |
| custom    | train            | 2      | 0.518             | 0.519    |
| custom    | train            | 3      | 0.518             | 0.525    |
| custom    | train_augmented  | 1      | 0.371             | 0.456    |
| custom    | train_augmented  | 2      | 0.458             | 0.469    |
| custom    | train_augmented  | 3      | 0.410             | 0.459    |

Таблица 9.7 — сводная статистика по трем запускам mlp

| Эмбединги | Обучающие данные | Balanced accuracy (mean $\pm$ std) | Macro F1 (mean $\pm$ std) |
|-----------|------------------|------------------------------------|---------------------------|
| default   | train            | 0.520 $\pm$ 0.025                  | 0.514 $\pm$ 0.003         |
| default   | train_augmented  | 0.447 $\pm$ 0.043                  | 0.476 $\pm$ 0.035         |
| custom    | train            | 0.511 $\pm$ 0.011                  | 0.529 $\pm$ 0.012         |
| custom    | train_augmented  | 0.413 $\pm$ 0.044                  | 0.461 $\pm$ 0.007         |

Лучшие средние метрики MLP получаются на исходном train: макро F1 = 0.529 на дообученных (custom) эмбедингах и 0.514 на baseline (default). Macro F1 между запусками довольно стабилен, а у balanced ассурасу дисперсия сильнее — то есть общий уровень качества воспроизводится, но распределение ошибок между частыми и редкими классами меняется. На train\_augmented качество в среднем падает, а разброс растёт, что согласуется с другими ветками: здесь аугментация скорее размывает границы классов и делает обучение менее стабильным. В целом MLP на композитных признаках не даёт выраженного преимущества над линейными моделями на том же входе: возможный выигрыш сопоставим с дисперсией между запусками.

### 9.6.2 Классические методы

Далее сравниваются линейные модели (LinearSVC, LogisticRegression, RidgeClassifier) на двух эмбеддерах (default и custom), двух train-наборах (train и train\_augmented) и трёх источниках признаков: bert\_only, tfidf\_only и hybrid. В таблицу вынесены лучшие варианты по макро F1 при подборе регуляризации, с использованием balanced-взвешивания классов.

Таблица 9.8 — классические методы на композитных представлениях

| Эмбединги | Обучающие данные | Источник признаков | Метод              | Balanced accuracy | Macro F1 |
|-----------|------------------|--------------------|--------------------|-------------------|----------|
| default   | train            | bert_only          | LinearSVC          | 0.333             | 0.317    |
| default   | train            | bert_only          | LogisticRegression | 0.318             | 0.292    |
| default   | train            | bert_only          | RidgeClassifier    | 0.361             | 0.288    |
| default   | train            | tfidf_only         | LinearSVC          | 0.529             | 0.521    |
| default   | train            | tfidf_only         | LogisticRegression | 0.513             | 0.467    |
| default   | train            | hybrid             | LogisticRegression | 0.442             | 0.451    |
| default   | train            | hybrid             | RidgeClassifier    | 0.559             | 0.532    |
| default   | train_augmented  | bert_only          | LinearSVC          | 0.403             | 0.373    |
| default   | train_augmented  | tfidf_only         | LogisticRegression | 0.458             | 0.430    |
| default   | train_augmented  | hybrid             | RidgeClassifier    | 0.514             | 0.486    |
| custom    | train            | bert_only          | LinearSVC          | 0.540             | 0.528    |
| custom    | train            | bert_only          | LogisticRegression | 0.534             | 0.510    |
| custom    | train            | bert_only          | RidgeClassifier    | 0.531             | 0.523    |
| custom    | train            | tfidf_only         | RidgeClassifier    | 0.521             | 0.508    |
| custom    | train            | hybrid             | LogisticRegression | 0.538             | 0.529    |
| custom    | train            | hybrid             | LinearSVC          | 0.526             | 0.541    |
| custom    | train_augmented  | bert_only          | LinearSVC          | 0.442             | 0.462    |
| custom    | train_augmented  | hybrid             | LinearSVC          | 0.446             | 0.463    |

Здесь проявляются ключевые закономерности. TF-IDF сам по себе даёт сильную базу: на `tfidf_only` макро F1 держится примерно на уровне 0.50–0.52 и, по построению, почти не зависит от того, какой эмбеддер используется. BERT-only без дообучения заметно слабее (макро F1 0.288–0.317), но после доменной донастройки эмбеддера поднимается до 0.510–0.528 — прирост порядка 0.21–0.24 наблюдается на всех трёх классификаторах, то есть эффект идёт именно от качества представления. Абсолютный максимум по макро F1 достигается на композите: LinearSVC на `custom + train + hybrid` даёт 0.541; при этом выигрыш над лучшим `bert_only` (0.528) и лучшим `tfidf_only` (0.521) небольшой (0.013 и 0.020), но композит важен тем, что он устойчивее к «слабым» baseline-эмбедингам — TF-IDF-часть частично страхует модель. Среди линейных моделей чаще всего лидируют LinearSVC и RidgeClassifier, а LogisticRegression заметно уступает в ключевых конфигурациях.

Эффект аугментации в этой серии в среднем отрицательный: в сильных конфигурациях (`custom` и/или `hybrid`) метрики падают, причём особенно заметно в лучшей точке (`custom + hybrid + LinearSVC`: 0.541 → 0.463). Единственное явное исключение — `default + bert_only + LinearSVC`, где аугментация даёт прирост (0.317 → 0.373): когда представление слабое и в `train` есть классы из 1 примера, синтетика действительно может помочь. Итог по разделу простой: лучший результат достигается при композитном представлении и дообученном эмбеддере, но основной «скелет» качества даёт TF-IDF, а BERT становится значимым усилителем только после доменной адаптации; аугментация полезна лишь как «костыль» для слабого BERT-only и мешает, когда признаки уже сильные.

### 9.7 Сводная таблица результатов лучших реализованных подходов

В этом разделе собраны абсолютные лидеры среди всех реализованных подходов: классические линейные методы (LinearSVC, LogisticRegression, RidgeClassifier), MLP-классификатор и косинусные методы (Cosine-Centroid, Cosine-Nearest, Cosine-CentroidNN). Каждая строка в сводных таблицах соответствует лучшей найденной конфигурации внутри конкретного подхода на заданной комбинации (эмбединги, обучающие данные, источник признаков). Поскольку внутри каждого семейства конфигурация выбирается отдельно под целевую метрику, одна и та же модель может присутствовать в обеих таблицах как «лучшая», но при разных настройках.

Для компактности в таблицах 9.9–9.10 используются сокращённые обозначения метрик: `accuracy` = `balanced accuracy`, `F1` = `macro F1`.

Таблица 9.9 — Сводная таблица результатов лучших подходов по balanced accuracy

| Подход       | Эмбединги | Данные | Источник   | Метод              | accuracy | F1    |
|--------------|-----------|--------|------------|--------------------|----------|-------|
| Косинусные   | custom    | train  | bert_only  | Cosine-Centroid    | 0.578    | 0.508 |
| Классические | custom    | train  | bert_only  | LogisticRegression | 0.573    | 0.490 |
| Классические | default   | train  | tfidf_only | LogisticRegression | 0.566    | 0.424 |
| Классические | custom    | train  | tfidf_only | LogisticRegression | 0.566    | 0.424 |
| Классические | default   | train  | tfidf_only | LinearSVC          | 0.561    | 0.497 |

Таблица 9.10 — Сводная таблица результатов лучших подходов по macro f1

| Подход       | Эмбединги | Данные | Источник  | Метод              | accuracy | F1    |
|--------------|-----------|--------|-----------|--------------------|----------|-------|
| MLP          | custom    | train  | hybrid    | MLP-StrongMLP      | 0.498    | 0.543 |
| Классические | custom    | train  | hybrid    | LinearSVC          | 0.526    | 0.541 |
| Классические | default   | train  | hybrid    | RidgeClassifier    | 0.559    | 0.532 |
| Классические | custom    | train  | hybrid    | LogisticRegression | 0.538    | 0.529 |
| Классические | custom    | train  | bert_only | LinearSVC          | 0.540    | 0.528 |

Сводные результаты показывают, что выбор лидера зависит от метрики и фактически отражает разные приоритеты качества. По balanced accuracy первое место занимает Cosine-Centroid на дообученных эмбедингах (0.578, при F1 = 0.508). Это означает, что после доменной адаптации эмбеддер формирует достаточно «структурированное» пространство, где центроиды классов становятся информативными, а простой метрический метод хорошо покрывает классы по полноте. В числе лидеров по balanced accuracy также остаются линейные модели на bert\_only и tfidf\_only, что дополнительно подчёркивает силу как доменно дообученных эмбедингов, так и лексических признаков TF-IDF.

По macro F1 картина иная: абсолютный максимум всей работы достигается на композитном представлении TF-IDF + BERT с дообученным эмбеддером. Лучший результат показывает MLP на custom + train + hybrid: F1 = 0.543 (accuracy = 0.498). Практически вплотную следует LinearSVC на той же комбинации: F1 = 0.541 (accuracy = 0.526). Разница между MLP и LinearSVC минимальна, поэтому для прикладного использования LinearSVC выглядит особенно привлекательным: он проще, быстрее обучается и обычно стабильнее, при почти максимальном качестве. Отдельно показательно, что высокие значения F1 достигаются именно на hybrid (и частично на bert\_only) при

custom-эмбеддере, то есть доменная адаптация и композитное пространство дают наиболее сбалансированный результат по точности и полноте.

## 9.8 Выводы по результатам экспериментального исследования

Эксперименты показали, что в задаче классификации длинных деловых писем ключевую роль играет качество признакового пространства, а не усложнение классификатора. TF-IDF на словных и символьных n-граммах формирует сильную базовую линию, потому что классы в деловой переписке часто различаются по лексическим маркерам: терминам, аббревиатурам, названиям проектов и типовым формулировкам. Поэтому линейные модели на TF-IDF остаются конкурентоспособными и входят в число сильных решений.

BERT-эмбединги без доменной адаптации в режиме `bert_only` оказываются слабее TF-IDF: универсальное представление «из коробки» недостаточно хорошо разделяет узкодоменные корпоративные классы. Однако после дообучения эмбеддера (`custom`) качество эмбедингов существенно улучшается: классы становятся лучше разделимыми, и `bert_only` начинает работать на уровне сильных классических признаков. Это подтверждает, что доменная адаптация является необходимым условием эффективного использования плотных контекстных представлений в данной задаче.

Наилучший результат по `macro F1` достигается при композитном (`hybrid`) представлении, объединяющем TF-IDF и дообученные BERT-эмбединги: максимум  $F1 = 0.543$  у MLP и почти идентичный результат  $F1 = 0.541$  у LinearSVC на `custom + train + hybrid`. Минимальный разрыв между нелинейной и линейной моделью показывает, что при сильных признаках усложнение классификатора даёт лишь маргинальный прирост, а простая линейная модель может быть практически оптимальной.

При этом разные метрики выделяют разные «лучшие» подходы. Если цель — более равномерная полнота по классам (`balanced accuracy`), лидером становится Cosine-Centroid на дообученных эмбедингах (0.578). Если цель — сбалансировать `precision` и `recall` по всем классам (`macro F1`), лидируют `hybrid`-подходы с custom-эмбеддером (MLP/LinearSVC). Следовательно, итоговый выбор модели должен определяться прикладным приоритетом: минимизация пропусков редких классов или более ровный общий баланс качества предсказаний.

Аугментация и суммаризация показали, что они не являются универсальными улучшениями. Суммаризация как замена исходного текста обычно снижает качество из-за потери важных деталей, а аугментация способна помогать в слабых конфигурациях, но в сильных представлениях чаще размывает границы классов и ухудшает метрики. В итоге наиболее надёжной стратегией оказывается сочетание качественной очистки, сильного TF-IDF-блока и доменно адаптированного эмбеддера, объединённых в композитное пространство.

## ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе была исследована задача многоклассовой классификации длинных русскоязычных деловых писем. В качестве исходных данных использовался корпоративный датасет из 36 классов, характеризующийся выраженным дисбалансом, значительной вариативностью длины документов и наличием доменно специфичной лексики. В ходе работы был реализован полный исследовательский и прикладной пайплайн: анализ данных, очистка и нормализация, аугментация и суммаризация, построение TF-IDF, BERT и композитных представлений, обучение и сравнение моделей на фиксированном train/test-разбиении.

Было показано, что качество предобработки и выбор признаков оказывают определяющее влияние на итоговые метрики. TF-IDF-представление на n-граммах формирует сильную базовую линию и остаётся конкурентоспособным по сравнению с нейросетевыми подходами, поскольку лексические маркеры деловой переписки несут существенный классовый сигнал. Контекстные BERT-эмбединги без доменной адаптации оказываются недостаточно разделяющими классы, однако после дообучения эмбеддера (custom) качество плотных представлений существенно возрастает, что делает доменную адаптацию ключевым условием их эффективного применения.

Наилучший результат по метрике macro F1 достигается на композитном представлении TF-IDF + дообученный BERT: максимальное значение  $F1 = 0.543$  показал MLP-классификатор на комбинации custom + train + hybrid, а LinearSVC на той же комбинации достиг  $F1 = 0.541$ , практически не уступая лидеру и оставаясь более простым и устойчивым решением для практического использования. По balanced accuracy лидером стал Cosine-Centroid на дообученных эмбедингах (0.578), что делает его перспективным вариантом в сценариях, где критично равномерно «находить» классы, включая редкие категории.

Аугментация и суммаризация в ходе экспериментов проявили себя как методы с контекстно-зависимой полезностью: они могут помогать в отдельных слабых конфигурациях, но в сильных представлениях нередко ухудшают качество за счёт размывания границ классов и потери доменных деталей. В совокупности результаты работы показывают, что наиболее эффективной стратегией для классификации деловой переписки является комбинация качественной очистки, сильного лексического блока (TF-IDF) и доменно адаптированных контекстных представлений (BERT) в композитном пространстве, при этом часто достаточно простого линейного классификатора.

Практическим результатом работы является модульный прототип классификационного пайплайна, пригодный для адаптации в задачах автоматической маршрутизации корпоративных писем и категоризации внутренних документов. Перспективы дальнейшего развития включают расширение обучающей выборки (особенно редких классов), более точечную аугментацию с сохранением доменной лексики, анализ ошибок по классам, а также исследование специализированных подходов для длинных документов и обучаемых механизмов объединения признаков.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

### Статьи и материалы конференций:

1. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser L., Polosukhin I. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — URL: <https://www.semanticscholar.org/paper/204e3073870fae3d05bcbc2f6a8e263d9b72e776>.
2. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // NAACL-HLT. — 2019. — P. 4171–4186. — URL: <https://aclanthology.org/N19-1423>.
3. Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L., Stoyanov V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. — 2019. — URL: <https://www.semanticscholar.org/paper/077f8329a7b6fa3b7c877a57b81eb6c18b5f87de>.
4. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // EMNLP-IJCNLP. — 2019. — P. 3982–3992. — URL: <https://aclanthology.org/D19-1410>.
5. Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language. — 2019. — URL: <https://arxiv.org/abs/1905.07213>.
6. Malykh V., et al. A Family of Pretrained Transformer Language Models for Russian. — 2023. — URL: <https://arxiv.org/abs/2309.10931>.
7. Beltagy I., Peters M.E., Cohan A. Longformer: The Long-Document Transformer. — 2020. — URL: <https://www.semanticscholar.org/paper/925ad2897d1b5decbea320d07e99afa9110e09b2>.
8. Wei J., Zou K. EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks // EMNLP-IJCNLP. — 2019. — P. 6382–6388. — URL: <https://aclanthology.org/D19-1670>.
9. Karimi A., Rossi L., Prati A. AEDA: An Easier Data Augmentation Technique for Text Classification // Findings of EMNLP. — 2021. — P. 2748–2754. — URL: <https://aclanthology.org/2021.findings-emnlp.234>.

10. Ciolino M., Noever D., Kalin J. Back Translation Survey for Improving Text Augmentation. — 2021. — URL: <https://www.semanticscholar.org/paper/34e6f0b17a2d413a68195692d3e35490a4e8a455>.
11. Jiang Z., Gao B., He Y., Han Y., Doyle P., Zhu Q. Text Classification Using Novel Term Weighting Scheme-Based Improved TF-IDF for Internet Media Reports // Mathematical Problems in Engineering. — 2021. — URL: <https://www.hindawi.com/journals/mpe/2021/6619088/>.
12. Watanangura P., Vanichrudee S., Minteer O., Sringamdee T., Thanngam N., Siriborvornratanakul T. A Comparative Survey of Text Summarization Techniques // SN Computer Science. — 2023. — URL: <https://link.springer.com/10.1007/s42979-023-02343-6>.
13. Myla S.D., Saini R., Kapoor N. Auto Text Summarization in Natural Language Processing: Review // IDCIoT. — 2024. — URL: <https://ieeexplore.ieee.org/document/10467376/>.
14. Taskiran S.F., Turkoglu B., Kaya E., Aşuroğlu T. A comprehensive evaluation of oversampling techniques for enhancing text classification performance // Scientific Reports. — 2025. — URL: <https://www.nature.com/articles/s41598-025-05791-7>.
15. Hinojosa Lee M.C., Braet J., Springael J. Performance Metrics for Multilabel Emotion Classification: Comparing Micro, Macro, and Weighted F1-Scores // Applied Sciences. — 2024. — URL: <https://www.mdpi.com/2076-3417/14/21/9863>.

### **Книги и учебные издания:**

1. Грон А. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных. — СПб.: Питер, 2020. — 400 с.
2. Жуков Л.А. Машинное обучение и анализ данных. — М.: МФТИ, 2019. — 520 с.
3. Jurafsky D., Martin J.H. Speech and Language Processing. 3rd ed. — Draft. — 2023. — URL: <https://web.stanford.edu/~jurafsky/slp3/>.
4. Goldberg Y. Neural Network Methods for Natural Language Processing. — San Rafael: Morgan & Claypool, 2017. — 310 p.

### **Электронные ресурсы и документация:**

1. DeepPavlov/rubert-base-cased: model card [Электронный ресурс]. — URL: <https://huggingface.co/DeepPavlov/rubert-base-cased> (дата обращения: 17.05.2026).
2. DeepPavlov/rubert-base-cased-sentence: model card [Электронный ресурс]. — URL: <https://huggingface.co/DeepPavlov/rubert-base-cased-sentence> (дата обращения: 17.05.2026).
3. ai-forever/ruRoberta-large: model card [Электронный ресурс]. — URL: <https://huggingface.co/ai-forever/ruRoberta-large> (дата обращения: 17.05.2026).
4. BERT in DeepPavlov documentation [Электронный ресурс]. — URL: <https://docs.deeppavlov.ai/en/master/features/models/bert.html> (дата обращения: 17.05.2026).
5. scikit-learn: TfidfVectorizer documentation [Электронный ресурс]. — URL: [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html) (дата обращения: 17.05.2026).
6. scikit-learn: LinearSVC documentation [Электронный ресурс]. — URL: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html> (дата обращения: 17.05.2026).
7. scikit-learn: LogisticRegression documentation [Электронный ресурс]. — URL: [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html) (дата обращения: 17.05.2026).

### **Нормативные документы:**

1. ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2018. — 24 с.
2. ГОСТ Р 7.0.5—2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — М.: Стандартинформ, 2008. — 18 с.

3. ГОСТ 7.1—2003. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2003. — 47 с.